

Efficiency for Neural Inference and Compression: Quantization, Sparsity, Distillation, Low-Rank Structure, and Benchmarking

March 2026

Abstract

This note explains inference optimization through a two-layer structure. Part I introduces the main ideas with a single running example, short calculations, and checkpoint summaries that keep the deployment bottleneck in view. Part II revisits the same topics with a unified optimization framework, fuller derivations, and a paper-by-paper discussion of historically important and recent influential work. The main lesson is simple but easy to miss: *an optimization is only useful if it improves the actual bottleneck on the target hardware and runtime stack*. Lower parameter count or lower theoretical FLOPs alone do not guarantee lower wall-clock latency.

How to use this note

First-pass reading plan

If you are new to the area, read **Part I only** on the first pass. It is written to be self-contained. In Part I, every major section ends with a short checkpoint box that asks and answers the most important questions. When you are comfortable with the ideas, move to Part II:

- For full PTQ/QAT derivations, go to Section 14.
- For OBD/OBS, structured sparsity, and LLM pruning, go to Section 15.
- For KD, MiniLLM, GKD, and low-rank adaptation, go to Section 16.
- For correct benchmarking and runtime methods such as FlashAttention, speculative decoding, and PagedAttention, go to Section 17.

What this note tries to optimize for

Many surveys are mathematically correct but pedagogically compressed. This note tries to avoid that failure mode. Part I repeats the same running example across sections, defines recurring terms in plain language, states the purpose of each formula before using it, and separates the “what is the idea?” lane from the “what is the full derivation?” lane.

Contents

I	Core ideas and deployment intuition	7
1	Minimal deep-learning and LLM primer	7
1.1	What a neural network computes	7
1.2	Logits, softmax, and cross-entropy	7
1.3	Training versus inference	8
1.4	What a transformer computes	8
1.5	How autoregressive generation works	9
2	A tiny math toolkit before the methods	9
3	What efficiency means in deployment	9
3.1	Five quantities to keep separate	10
3.2	Why fewer FLOPs do not automatically mean lower latency	10
3.3	A tiny serving example	11
3.4	Where the four model-side methods fit	11
4	A single running example for all four methods	12
4.1	Quantization on the running example	12
4.2	Pruning on the running example	12
4.3	Low-rank structure on the running example	13
4.4	Distillation on the running example	13
5	The four efficiency levers at a glance	13
6	Quantization	14
6.1	What quantization is in plain language	14
6.2	A scalar quantization example by hand	14
6.3	The affine quantization formula	15
6.4	Why integer-only inference is important	16
6.5	PTQ versus QAT	16
6.6	How PTQ is judged: keep outputs similar	17

6.7	How QAT is judged: learn under fake quantization	17
6.8	Common numerical pitfalls	17
7	Pruning and sparsity	18
7.1	Three kinds of sparsity	19
7.2	The simplest pruning rule: remove small weights	19
7.3	Why magnitude is only a rough proxy	19
7.4	Why unstructured sparsity often fails to speed up GPUs	20
7.5	The group-sparsity idea	20
7.6	When pruning gives real speedups	20
8	Distillation for deployable models	21
8.1	Hard labels versus soft labels	21
8.2	The basic KD loss	21
8.3	Why distillation can beat naive small-model training	22
8.4	Feature distillation and sequence-level distillation	22
8.5	Why distillation matters especially for deployment	22
9	Low-rank structure and fast adaptation	23
9.1	What “rank” means in plain language	23
9.2	A simple rank-1 picture	23
9.3	The best low-rank approximation idea	23
9.4	Why LoRA became so important	24
9.5	Quantized backbones: QLoRA and related methods	24
10	Benchmarking and runtime optimization	24
10.1	Latency, throughput, concurrency, and percentiles	25
10.2	A simple mental experiment	25
10.3	Why prefill and decode should be measured separately	25
10.4	Three runtime ideas to keep in mind	25
10.5	A correct measurement protocol in one list	25
11	What to try first in practice	26
II	Rigorous survey and literature map	28

12 Notation and deployment view for the rigorous survey	28
13 A unified mathematical framework for efficiency	28
13.1 Deployment-constrained optimization	29
13.2 Local quadratic view: why second-order structure keeps reappearing	30
13.3 Runtime lower bounds and the bottleneck principle	30
13.4 A short decision rule	31
14 Quantization: basics, PTQ vs QAT, pitfalls, and recent work	31
14.1 First-pass picture: what quantization changes	31
14.2 Quantization as constrained approximation	32
14.3 How the scale and zero-point are obtained	32
14.4 The historically important deployment bridge: integer-only inference	33
14.5 From scalar quantization to layer quantization	33
14.6 PTQ vs QAT at a high level	33
14.7 Canonical PTQ derivation: from output reconstruction to GPTQ-style compensation	34
14.8 Canonical QAT derivation: fake quantization and the straight-through estimator . .	36
14.9 Historical derivation snapshot: PACT and learnable clipping	37
14.10 Historical derivation snapshot: why Hessian trace drives mixed precision	37
14.11 Historical derivation snapshot: AdaRound and BRECQ	38
14.12 Numerical pitfalls in quantization	39
14.13 How to evaluate accuracy drop correctly	39
14.14 Historical lineage	40
14.15 Recent influential works and how they extend the canonical PTQ/QAT framework .	40
15 Pruning and sparsity: unstructured vs structured, and when speedups are real	45
15.1 Pruning as constrained optimization	45
15.2 Detailed derivation of the classical unstructured framework: OBD, magnitude pruning, and OBS	46
15.3 Canonical structured pruning framework: group sparsity and its proximal step . . .	48
15.4 Historical derivation snapshot: network slimming and channel saliency	48
15.5 Unstructured vs structured vs semi-structured sparsity	49
15.6 When sparsity yields real speedups	49
15.7 Historical lineage	50

15.8	Recent influential works and their relation to the classical framework	50
16	Distillation for deployable models, then low-rank structure and fast adaptation	53
16.1	First-pass picture: what distillation changes	53
16.2	Canonical knowledge distillation objective	54
16.3	Why the temperature-squared factor appears: full gradient derivation	55
16.4	Historical derivation snapshot: FitNets and intermediate feature regression	55
16.5	Historical derivation snapshot: relation distillation in transformers	55
16.6	Sequence-level distillation and the train-inference mismatch	56
16.7	Reverse KL and MiniLLM’s generative view	56
16.8	Generalized knowledge distillation (GKD) and on-policy learning	57
16.9	How to evaluate a distilled model for deployment	57
16.10	Low-rank compression: the Eckart-Young derivation	58
16.11	LoRA as constrained low-rank adaptation	58
16.12	Fast adaptation on quantized backbones: QLoRA and LoftQ	59
16.13	Adaptive rank allocation and weight decomposition: AdaLoRA and DoRA	59
16.14	Historically important works from root to current practice	60
16.15	Recent influential works	60
17	Benchmarking latency and throughput correctly	62
17.1	The right metrics	62
17.2	Latency decomposition and why averages are not enough	63
17.3	Roofline and queueing: the two views every benchmark should include	63
17.4	IO-aware runtime optimization: FlashAttention as a worked example	63
17.5	Speculative decoding: fewer target-model passes without changing the target distribution	64
17.6	PagedAttention and continuous batching: why KV-cache management changes throughput	65
17.7	A correct measurement protocol	65
17.8	Common measurement fallacies	66
17.9	Recent influential works on measurement and serving	66
18	A practical synthesis: what usually works in deployment	68
19	Conclusion	69

A	Historical lineage at a glance	70
B	Glossary and FAQ	71
B.1	Glossary	71
B.2	FAQ	72

Part I

Core ideas and deployment intuition

1 Minimal deep-learning and LLM primer

This section is the only prerequisite needed for the rest of Part I. Nothing here assumes previous exposure to neural-network compression or systems papers.

1.1 What a neural network computes

At a coarse level, a neural network repeatedly does two things.

Step 1: linear mixing. A linear layer takes an input vector x and computes

$$h = Wx + b.$$

The matrix W contains the learned weights; b is a bias. The output h is a new vector whose entries are weighted sums of the entries of x .

Step 2: nonlinearity. The model then applies an elementwise function such as ReLU, GELU, or SiLU. If we write the nonlinearity as $\sigma(\cdot)$, a simple hidden layer is

$$u = \sigma(Wx + b).$$

A deep network stacks many such layers. So, from the efficiency point of view, a large fraction of inference is just a long sequence of matrix multiplications and elementwise operations.

Intuition

If you remember only one sentence from this subsection, remember this: *most model-side efficiency methods work by changing either the matrices, the precision of the numbers stored in those matrices, or the way those matrices are executed on hardware.*

1.2 Logits, softmax, and cross-entropy

For classification, the last layer usually produces a vector of *logits* $z \in \mathbb{R}^K$, one score per class. To turn logits into probabilities, we apply softmax:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}.$$

If class i has the largest logit, it gets the largest probability.

Suppose the correct label is represented by a one-hot vector y . Then the standard training loss is

the *cross-entropy*

$$\text{CE}(y, p) = - \sum_{i=1}^K y_i \log p_i.$$

Because y is one-hot, this just means “take minus the log-probability of the correct class.” The loss is small when the model places high probability on the correct answer.

Common confusion

The model does *not* output probabilities directly. It first outputs logits. Softmax then turns logits into probabilities. Distillation works largely by changing what probability targets the student tries to match.

1.3 Training versus inference

Training means updating parameters so the loss becomes small on data. Training uses backpropagation and gradient-based optimization, so it is expensive.

Inference means applying a fixed trained model to new inputs. No learning happens. Inference is cheaper than training for one example, but a deployed service may perform inference millions of times. That is why even a small per-request speedup matters.

The central design question of this note is therefore:

How can we make inference cheaper without hurting prediction quality too much?

1.4 What a transformer computes

Modern language models are usually transformers. On a first pass, you do not need every detail immediately, but four blocks matter:

- **Embedding:** maps a token index to a vector.
- **Attention:** lets each token use information from other tokens.
- **Feed-forward network (FFN):** another stack of linear layers and nonlinearities.
- **Residual and normalization layers:** stabilize learning and allow deep networks to train.

For one attention head, the model forms three matrices from the input hidden states H :

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V.$$

The output is roughly

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

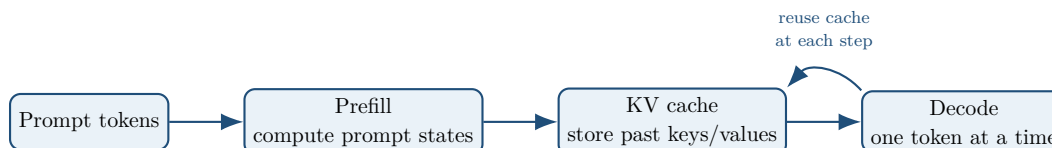
You do not need to memorize this formula yet. The important point is that attention involves large matrix multiplications and moving large tensors in and out of memory. That is why both model compression and runtime optimization matter.

1.5 How autoregressive generation works

An LLM generates text one token at a time. There are two phases.

Prefill. The model processes the input prompt. This stage fills the *KV cache*, which stores past keys and values so they do not need to be recomputed for every future token.

Decode. The model generates new tokens one by one. At each step it reuses the KV cache, computes the next-token distribution, samples or chooses a token, appends it, and repeats.



Why does this matter for efficiency? Because prefill and decode stress the hardware differently. Prefill often benefits from batching and high arithmetic throughput. Decode often becomes bandwidth-bound because many model weights and cached activations must be read for each generated token.

Checkpoint: what should you understand so far?

- A neural network is largely a sequence of matrix multiplications and elementwise operations.
- Logits become probabilities only after softmax.
- Training changes parameters; inference uses fixed parameters.
- An LLM has a prompt-processing phase (prefill) and a token-by-token generation phase (decode).
- The KV cache is the memory of past keys and values used during decode.

If any of these ideas still feel unclear, pause here. The rest of the survey assumes them.

2 A tiny math toolkit before the methods

Part I uses only six mathematical ideas again and again. You do not need more than this table on a first read.

Do not panic about the symbols

You do *not* need to master this table before reading on. It is only here so that symbols later in the note never arrive without a home.

3 What efficiency means in deployment

The word “efficient” is overloaded. In research papers it can mean fewer parameters, fewer FLOPs, lower energy, shorter latency, or larger throughput. These are related but not identical.

Table 1: The six ideas that reappear throughout the survey.

Idea	Formula	Why it matters here
Layer output	$y = Wx + b$	Almost every efficiency method changes W , the precision of its entries, or how the product is executed.
Reconstruction error	$\ WX - \hat{W}X\ _F^2$	PTQ and some pruning methods keep outputs of the modified layer close to those of the original layer.
Quadratic approximation	$\mathcal{L}(\theta + \delta) \approx \mathcal{L}(\theta) + g^\top \delta + \frac{1}{2} \delta^\top H \delta$	Near a trained solution, curvature explains why second-order methods keep appearing.
KL divergence	$\text{KL}(P\ Q) = \sum_i P_i \log \frac{P_i}{Q_i}$	Distillation compares teacher and student probability distributions with KL-like objectives.
Low rank / SVD	$W = U\Sigma V^\top$	The singular values tell us which directions of a matrix matter most.
Little’s law	$L = \lambda W$	In serving systems, concurrency equals throughput times latency. This helps interpret benchmark operating points.

3.1 Five quantities to keep separate

Table 2: The five deployment quantities that matter most.

Quantity	Formal meaning	Plain-language reading
Model memory	Bytes needed to store the model weights and any metadata	“How large is the loaded model?”
KV-cache memory	Bytes needed for past keys and values during generation	“How much extra memory does long-context decoding consume?”
Latency	Wall-clock time for one request	“How long does one user wait?”
Throughput	Requests/s or tokens/s under a stated load	“How much work can the service finish per second?”
Energy	Joules/request or tokens/joule	“How much electrical cost is paid?”

3.2 Why fewer FLOPs do not automatically mean lower latency

Imagine two models on a GPU.

- Model A does more arithmetic but reads memory very efficiently.
- Model B does fewer arithmetic operations but jumps around memory and launches inefficient kernels.

Model B can have fewer theoretical FLOPs and still be slower in practice. This is the most important systems lesson of the whole note. Compression changes the model. Runtime optimization changes how the model is executed. Real speedups usually require both.

A useful mental model

For modern accelerators, the runtime of an operation is often limited by one of two things:

$$\text{time} \approx \max\left(\frac{\text{FLOPs}}{\text{peak compute}}, \frac{\text{bytes moved}}{\text{peak memory bandwidth}}\right).$$

If the second term dominates, reducing bytes moved can matter more than reducing FLOPs. This is why quantization often helps immediately and why some sparse models fail to speed up.

3.3 A tiny serving example

Suppose a server handles one LLM request with a 1000-token prompt and generates 200 tokens.

- **Time to first token (TTFT):** how long until the first generated token appears.
- **Time per output token (TPOT):** average time for each additional generated token.
- **End-to-end latency:** TTFT plus the total decode time.

Now suppose the server handles many requests at once. Throughput may rise because the hardware is better utilized, but latency for each request may also rise because requests compete for memory and compute. Therefore a benchmark must always state *the operating point*: prompt length, output length, batch size or concurrency, and hardware/software stack.

Common confusion

“Fast” for one user and “fast” for a datacenter are not the same thing. A single-stream latency benchmark answers “How quickly does one request finish?” A throughput benchmark answers “How much work can the system do per second under load?” They are different experiments.

3.4 Where the four model-side methods fit

The rest of Part I studies four levers:

- **Quantization:** store numbers with fewer bits.
- **Pruning / sparsity:** set some weights, channels, or sublayers to zero or remove them.
- **Distillation:** train a smaller student to imitate a larger teacher.
- **Low-rank structure / adaptation:** represent a weight matrix or weight update using fewer degrees of freedom.

All four trade model fidelity against deployment cost. They differ mainly in *what they change* and *when they require retraining*.

Checkpoint: the deployment view in one paragraph

Efficiency is not one number. Memory, latency, throughput, and energy are different targets. A paper that reduces parameters or FLOPs does *not* automatically reduce latency. The only honest question is: what bottleneck matters on the actual hardware and runtime stack, and does the method improve that bottleneck?

4 A single running example for all four methods

We will repeatedly return to the same tiny layer so that the ideas stay concrete:

$$W = \begin{bmatrix} 0.9 & -0.2 \\ 0.1 & 1.3 \end{bmatrix}, \quad x = \begin{bmatrix} 2 \\ -1 \end{bmatrix}.$$

Its output is

$$y = Wx = \begin{bmatrix} 2.0 \\ -1.1 \end{bmatrix}.$$

4.1 Quantization on the running example

Suppose we use a coarse symmetric grid with scale $s = 0.5$ and integer codes restricted to

$$q \in \{-3, -2, -1, 0, 1, 2, 3\}.$$

Then each weight is replaced by the nearest grid value:

$$\hat{W}_{\text{quant}} = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.5 \end{bmatrix}.$$

The output becomes

$$\hat{y}_{\text{quant}} = \hat{W}_{\text{quant}}x = \begin{bmatrix} 2.0 \\ -1.5 \end{bmatrix}.$$

The first coordinate stayed exact for this input; the second changed from -1.1 to -1.5 . Quantization saved memory, but it introduced approximation error.

4.2 Pruning on the running example

The smallest-magnitude entry is 0.1. If we prune it to zero,

$$\hat{W}_{\text{prune}} = \begin{bmatrix} 0.9 & -0.2 \\ 0.0 & 1.3 \end{bmatrix}, \quad \hat{y}_{\text{prune}} = \begin{bmatrix} 2.0 \\ -1.3 \end{bmatrix}.$$

The output changed only a little for this particular input.

4.3 Low-rank structure on the running example

A rank-1 matrix can always be written as $uv^\top$. If we approximate W by a rank-1 matrix, we keep only the strongest input-output interaction pattern and discard the weaker one. This may reduce the number of parameters or the cost of multiplying by W , but it introduces approximation error.

4.4 Distillation on the running example

Distillation is easier to understand at the *prediction* level than at the single-layer level. Imagine a teacher classifier outputs the probabilities

$$p_t = [0.70, 0.25, 0.05]$$

over three classes, while the hard label is just class 1. A smaller student can learn not only that class 1 is correct, but also that class 2 is much more plausible than class 3. That extra information often helps a smaller model learn better.

The point of the running example

Every method in this survey does one of two things:

- change the numbers or the structure inside the model while trying to keep outputs similar, or
- train a smaller model so it behaves similarly to a larger one.

The rest of Part I simply unpacks this sentence method by method.

5 The four efficiency levers at a glance

Table 3: A one-page comparison before we study the methods in detail.

Method family	What changes?	Why can it help?	Main risk or catch
Quantization	The numerical precision of weights, activations, or KV cache	Fewer bytes moved and stored; sometimes faster kernels	Rounding and clipping can hurt accuracy; speedups need kernel support
Pruning / sparsity	Some weights, channels, heads, or blocks are removed or set to zero	Smaller model or fewer useful operations	Unstructured zeros often fail to accelerate dense hardware
Distillation	The training target: a smaller student learns from a larger teacher	Better quality-latency trade-off when training a smaller model is allowed	Requires retraining and a teacher model; not a pure post-processing method
Low-rank structure	A matrix or update is represented by fewer factors or fewer important directions	Fewer trainable parameters or cheaper adaptation	Too small a rank can underfit; some forms help training more than inference

A historical map before the details

The field did not begin with today’s LLM papers. The broad lineage is:

1. early parameter redundancy and low-rank ideas in the 2013–2014 period;
2. classical pruning theory in OBD and OBS;
3. mainstream neural-network compression with Deep Compression and teacher-student learning;
4. integer-only and learned-step quantization for practical deployment;
5. LLM-era PTQ, QAT, sparse one-shot pruning, and quantized low-rank adaptation;
6. serving-side methods such as FlashAttention, speculative decoding, and PagedAttention.

Part II develops that lineage carefully from root to leaf.

6 Quantization

Quantization is often the first efficiency method people try because the memory win can be immediate and the workflow can be simple.

6.1 What quantization is in plain language

A floating-point weight may use 16 or 32 bits. Quantization replaces it by a lower-precision representation such as 8 bits or 4 bits. If a model is large, the memory savings can be dramatic.

For example, if a model uses 32-bit weights and we change to 8-bit weights, the raw weight storage drops by roughly a factor of four. If the model was limited by memory bandwidth, this can also help latency.

But note the word “raw” in the previous sentence

Real systems store more than just integer codes. They may also store scales, zero-points, group metadata, or dequantization buffers. So the achieved memory reduction is slightly less than the ideal ratio.

6.2 A scalar quantization example by hand

Take one weight value:

$$w = 0.93.$$

Choose a symmetric scale $s = 0.25$ and integer range $q \in \{-7, -6, \dots, 7\}$. Quantization proceeds in three steps.

Step 1: divide by the scale.

$$w/s = 0.93/0.25 = 3.72.$$

Step 2: round to the nearest integer code.

$$q = \text{round}(3.72) = 4.$$

Step 3: map back to a representable real number.

$$\hat{w} = sq = 0.25 \times 4 = 1.00.$$

So the original value 0.93 becomes 1.00. The quantization error is

$$\hat{w} - w = 0.07.$$

Now take a larger value $w = 2.3$. We again divide by the scale:

$$2.3/0.25 = 9.2.$$

But 9.2 is outside the allowed integer range $[-7, 7]$. So we *clip* to 7 and get

$$\hat{w} = 0.25 \times 7 = 1.75.$$

This error is much larger. This is why outliers are a major problem in low-bit quantization.

6.3 The affine quantization formula

The most common formula is

$$x \approx s(q - z),$$

where

- q is the stored integer code,
- $s > 0$ is the scale,
- z is the zero-point.

Why is this useful? Because it says the real value x can be reconstructed from a small integer q plus a small amount of metadata.

There are two common special cases.

Symmetric quantization. Set $z = 0$. Then

$$x \approx sq.$$

This is simple and often hardware-friendly.

Asymmetric quantization. Use a nonzero z so that zero in real space is represented exactly even when the dynamic range is not symmetric around zero. This is common for activations.

If the calibration range is $[x_{\min}, x_{\max}]$ and the integer codes run from 0 to $Q_{\max}$, the simplest

min-max choice is

$$s = \frac{x_{\max} - x_{\min}}{Q_{\max}}, \quad z = \text{round}\left(-\frac{x_{\min}}{s}\right).$$

This formula is not the last word in quantization; it is just the most basic way to map a real interval to an integer interval.

6.4 Why integer-only inference is important

Suppose the activation vector and the weight vector are approximated by

$$x \approx s_x(q_x - z_x), \quad w \approx s_w(q_w - z_w).$$

Then their dot product is approximately

$$w^\top x \approx s_w s_x \sum_i (q_{w,i} - z_w)(q_{x,i} - z_x).$$

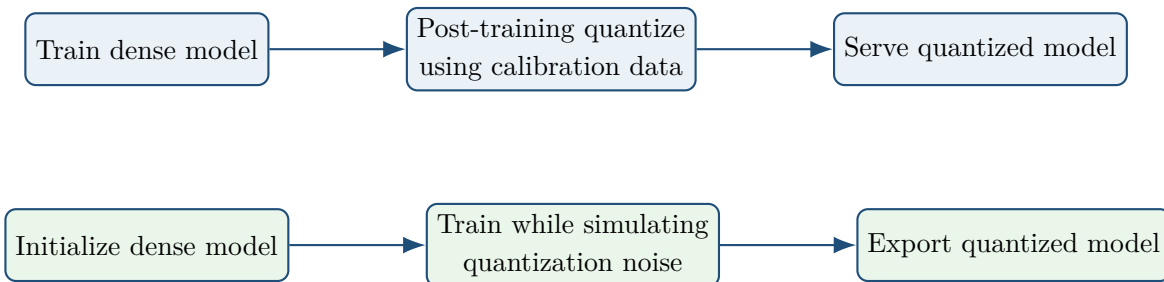
This matters because the sum on the right can be carried out using integer arithmetic, followed by a small amount of rescaling. Jacob et al. [5] turned this observation into the classical integer-only inference recipe used in many deployment stacks.

Why quantization often helps immediately

Even if the compute path is not perfectly optimized, a smaller weight representation means fewer bytes moved from memory. On modern accelerators that alone can be valuable.

6.5 PTQ versus QAT

There are two main workflows.



Post-training quantization (PTQ). First train the model normally. Then quantize it afterwards, usually using a small *calibration set* to estimate activation ranges or reconstruction error. PTQ is attractive because it is cheap and simple.

Quantization-aware training (QAT). Simulate quantization during training or fine-tuning. The model then adapts its weights to the noise introduced by rounding and clipping. QAT is more expensive than PTQ but usually performs better at very low precision.

Table 4: A first-pass PTQ/QAT comparison.

Question	PTQ	QAT
When is quantization applied?	After ordinary training has finished	During a dedicated training or fine-tuning stage
How much extra compute is needed?	Low	Moderate to high
When does it shine?	Fast deployment, large models, limited retraining budget	Very low bits, difficult activation distributions, tight accuracy budgets
What is the main idea?	Approximate the trained model well without changing it much	Let the model adapt to quantization noise while training

6.6 How PTQ is judged: keep outputs similar

The most common layer-level PTQ idea is very simple. We have a full-precision layer output

$$Y = WX$$

on a calibration batch X , and we look for a quantized weight matrix $\hat{W}$ that keeps the output similar:

$$\hat{W} \approx \arg \min \|WX - \hat{W}X\|_F^2.$$

Read this slowly: among all quantized weight matrices, find one that makes the layer output on typical inputs as close as possible to the original output. This reconstruction view is the backbone of many PTQ methods, including GPTQ-like methods discussed rigorously in Section 14.

6.7 How QAT is judged: learn under fake quantization

In QAT we insert a *fake quantization* operator during training:

$$\hat{x} = Q(x).$$

The forward pass uses $\hat{x}$, which behaves like a quantized value, but the model still stores a full-precision copy of the trainable parameters internally so gradients can update them. Because rounding is not differentiable, QAT usually uses a surrogate called the *straight-through estimator* (STE). Part II derives this carefully.

6.8 Common numerical pitfalls

1. **Outliers.** A few unusually large activations or weights can force a large scale and waste most quantization levels on empty range.
2. **Bad granularity choice.** Per-tensor quantization is simple but can be too coarse. Per-channel

or per-group quantization often works better.

3. **Calibration mismatch.** If the calibration set does not resemble deployment inputs, PTQ can choose the wrong scales.
4. **Kernel mismatch.** A numerically good quantization scheme may still be slow if the deployment runtime has no efficient kernel for it.
5. **Evaluation mismatch.** Accuracy drop should be measured on the real task, not only on layer reconstruction error.

Names to remember on a first read

On a first pass, four names are worth remembering:

- **Jacob et al.** for practical integer-only inference [5].
- **PACT** and **LSQ** for learning clipping or step size during QAT [6, 54].
- **GPTQ** for one-shot PTQ of LLM weights [9].
- **SmoothQuant, AWQ, OmniQuant, LLM-QAT, QuaRot, SpinQuant** for LLM-era activation handling, weight protection, calibration, data-free QAT, and learned rotations [11, 12, 60, 61, 13, 14].

Part II explains what each of these changed relative to the common framework.

Checkpoint: can you now answer these?

Question 1. What is the difference between PTQ and QAT?

Answer. PTQ quantizes *after* training and tries to approximate the trained model well. QAT trains or fine-tunes *with* quantization in the loop so the model can adapt to rounding and clipping noise.

Question 2. Why can quantization reduce memory but fail to reduce latency?

Answer. Because the runtime may not have an efficient kernel for the chosen precision or quantization layout. Then the system may spend time dequantizing or using inefficient memory access patterns.

Question 3. Why are outliers dangerous?

Answer. Because a few large values can force a large scale, leaving too few useful quantization levels for the many ordinary values near zero.

7 Pruning and sparsity

Pruning makes a model smaller by removing parameters or larger structural pieces. The word *sparsity* means that many entries are zero.

7.1 Three kinds of sparsity

Consider the dense matrix

$$\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix}.$$

Unstructured sparsity allows zeros anywhere:

$$\begin{bmatrix} \times & 0 & \times & 0 \\ 0 & \times & 0 & \times \\ \times & 0 & 0 & \times \\ 0 & 0 & \times & \times \end{bmatrix}.$$

Structured sparsity removes whole rows, columns, heads, or channels:

$$\begin{bmatrix} \times & \times & 0 & \times \\ \times & \times & 0 & \times \\ \times & \times & 0 & \times \\ \times & \times & 0 & \times \end{bmatrix}.$$

Semi-structured or $N:M$ **sparsity** imposes a local rule such as “in each group of 4 weights, only 2 may be nonzero.” This is more regular than unstructured sparsity and can map better to hardware.

7.2 The simplest pruning rule: remove small weights

The first idea many people try is magnitude pruning: weights with small absolute value are more likely to be removed. In our running example, 0.1 is the obvious first candidate.

Why can this work? Because if changing one weight barely changes the outputs or the loss, that weight is not very important. Magnitude is a rough proxy for importance.

7.3 Why magnitude is only a rough proxy

Suppose the loss around a trained point looks like

$$\mathcal{L}(w) \approx \mathcal{L}(w^*) + \frac{1}{2}h(w - w^*)^2$$

for one scalar weight. If we prune by setting w^* to zero, the loss increase is approximately

$$\Delta\mathcal{L} \approx \frac{1}{2}h(w^*)^2.$$

Two things matter:

- the weight magnitude $|w^*|$,
- the curvature h .

So a small weight in a very sensitive direction may matter more than a larger weight in a flat direction. This is the intuition behind second-order pruning methods such as OBD and OBS.

7.4 Why unstructured sparsity often fails to speed up GPUs

A dense GPU kernel is optimized to read contiguous blocks and do dense matrix multiplication. If half the entries are zero but the runtime still stores the matrix densely and uses the same dense kernel, it will still read almost all the same bytes. In that case, the zeros help memory footprint but not latency.

By contrast, if we prune an entire channel or block and physically shrink the matrix, the next layer also shrinks. That is why structured pruning has a clearer path to real speedups.

Rule of thumb

Unstructured sparsity is easier to obtain but harder to accelerate.

Structured sparsity is harder to obtain without hurting accuracy, but easier to accelerate because the problem size really gets smaller.

$N:M$ **sparsity** tries to sit in the middle: more flexible than structured pruning, more hardware-friendly than arbitrary zeros.

7.5 The group-sparsity idea

Suppose we want to prune whole channels or blocks. Then we should penalize or threshold *groups*, not individual weights. A common objective is

$$\mathcal{L}(W) + \lambda \sum_g \|W_g\|_2,$$

where W_g is one group, such as all weights in one output channel. The group norm encourages entire groups to go to zero together. Part II derives the proximal update that makes this work.

7.6 When pruning gives real speedups

Pruning is most likely to produce wall-clock gains when:

- the sparsity pattern matches hardware-supported kernels;
- the pruned structure actually reduces matrix sizes or memory traffic;
- the runtime avoids paying bookkeeping overhead larger than the saved work;
- the benchmark measures end-to-end latency or throughput, not just parameter count.

Names to remember on a first read

- **Optimal Brain Damage** and **Optimal Brain Surgeon** gave the classical second-order view of pruning [15, 16].
- **Learning both weights and connections** and **Deep Compression** made pruning a mainstream deployment tool [17, 18].
- **Network Slimming** helped popularize structured channel pruning [20].
- **SparseGPT** and **Wanda** brought one-shot pruning to modern LLMs [25, 26].

Checkpoint: the practical pruning lesson

Question 1. Why can a 50% sparse model fail to be 2x faster?

Answer. Because the runtime may still use dense kernels or pay large indexing overhead. Zeros alone do not guarantee faster execution.

Question 2. Why is structured pruning usually easier to accelerate?

Answer. Because it removes entire rows, columns, heads, or blocks, which actually shrinks the tensors and the subsequent dense computations.

Question 3. What is the limitation of plain magnitude pruning?

Answer. It ignores curvature. A small weight can still be important if the loss is very sensitive to that direction.

8 Distillation for deployable models

Distillation trains a smaller *student* model to imitate a larger *teacher*. Unlike PTQ or pruning, it is not mainly a post-processing step. It is a training strategy for building a better small model.

8.1 Hard labels versus soft labels

Suppose a teacher predicts

$$p_t = [0.70, 0.25, 0.05]$$

for three classes and the ground-truth label is class 1.

With **hard-label training**, the student sees only that class 1 is correct.

With **distillation**, the student also sees that class 2 is somewhat plausible and class 3 is not. This extra structure is often called *dark knowledge*. It tells the student more about the teacher’s view of the task than a one-hot label can.

8.2 The basic KD loss

The standard KD loss combines the usual supervised loss with a teacher-matching term:

$$\mathcal{L}_{\text{KD}} = (1 - \alpha) \text{CE}(y, p_s) + \alpha \tau^2 \text{KL}(p_t^\tau \| p_s^\tau).$$

Here:

- p_s is the student distribution,
- p_t is the teacher distribution,
- τ is the *temperature*,
- α balances the hard-label loss and the teacher-matching loss.

A higher temperature smooths the distributions, making relative class preferences easier to see. Part II derives why the factor τ^2 appears.

8.3 Why distillation can beat naive small-model training

A small model has limited capacity. If it trains only on hard labels, it may waste capacity learning boundaries from scratch. The teacher provides a richer target. Distillation can therefore give a better quality-latency tradeoff than simply shrinking the model and training normally.

8.4 Feature distillation and sequence-level distillation

There is more than one place to imitate the teacher.

Logit distillation. Match the teacher’s output distribution. This is the standard KD view.

Feature distillation. Match intermediate hidden representations. FitNets, TinyBERT, and MiniLM are examples of this line of thought [31, 34, 35].

Sequence-level distillation. For sequence generation, train the student on sequences generated by the teacher rather than only on the human reference [32]. This can reduce train-inference mismatch.

8.5 Why distillation matters especially for deployment

Distillation is valuable when you are allowed to train a new model family member for deployment. Instead of forcing a huge model to become cheap after the fact, you build a smaller model that is cheap by design but informed by a stronger teacher.

Names to remember on a first read

- **Bucilua et al.** and **Hinton et al.** established the modern teacher-student view [29, 30].
- **FitNets** popularized intermediate-feature guidance [31].
- **DistilBERT**, **TinyBERT**, **MiniLM** made transformer distillation practical [33, 34, 35].
- **MiniLLM** and **GKD** adapted distillation to generative LLMs with reverse-KL and on-policy ideas [39, 40].

Checkpoint: what does the teacher provide that the hard label does not?

The teacher provides a full probability distribution or even intermediate representations, not just a single correct class. This richer signal often helps the student allocate its limited capacity more wisely.

9 Low-rank structure and fast adaptation

Low-rank methods exploit the fact that many weight matrices or weight updates have redundancy.

9.1 What “rank” means in plain language

A matrix with low rank has only a few truly independent directions. A rank-1 matrix can be written as

$$W = uv^\top.$$

This means every column is a multiple of the same vector u , or every row is a multiple of the same vector $v^\top$. Such a matrix is highly structured.

Why is that useful? Because storing u and v can be cheaper than storing the whole matrix.

9.2 A simple rank-1 picture

Suppose

$$u = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad v^\top = \begin{bmatrix} 1 & -1 \end{bmatrix}.$$

Then

$$uv^\top = \begin{bmatrix} 2 & -2 \\ 1 & -1 \end{bmatrix}.$$

Instead of 4 independent numbers, this matrix is determined by the 4 entries of u and v . For larger matrices the savings can be substantial when the rank is small.

9.3 The best low-rank approximation idea

If a full matrix W is not exactly low-rank, we can still approximate it by the best rank- r matrix under a chosen error measure. The singular value decomposition (SVD) tells us how to do this. Part II proves the classical Eckart-Young result step by step.

9.4 Why LoRA became so important

LoRA does not try to compress the entire base model aggressively. Instead, it freezes the pretrained weight matrix W_0 and learns only a low-rank update:

$$W = W_0 + BA,$$

where $B \in \mathbb{R}^{m \times r}$ and $A \in \mathbb{R}^{r \times d}$ with small rank r .

This is attractive because:

- the number of trainable parameters becomes small;
- the base model can stay frozen;
- after training, the update BA can often be merged into W_0 for inference.

9.5 Quantized backbones: QLoRA and related methods

QLoRA goes one step further: keep the backbone quantized and still train low-rank adapters on top [42]. This greatly reduces memory during adaptation. LoftQ improves the starting point by making the quantized backbone and the low-rank adapter fit together better [43].

Compression versus adaptation

Low-rank methods appear in two different roles:

- **compression:** approximate a large weight matrix by a lower-rank one;
- **adaptation:** keep the large pretrained matrix fixed and learn only a low-rank update.

These are related but not identical use cases.

Names to remember on a first read

- **Denil et al.** and **Denton et al.** are part of the early low-rank compression story [36, 37].
- **LoRA** is the key modern low-rank adaptation method [38].
- **AdaLoRA**, **QLoRA**, **LoftQ**, and **DoRA** are important follow-ups for budget allocation, quantized backbones, initialization, and weight decomposition [44, 42, 43, 45].

Checkpoint: why can low-rank adaptation be cheap at inference?

Because the number of trainable adaptation parameters is small during training, and after training the learned update can often be merged into the base weights. In that merged case, there may be little or no extra inference overhead.

10 Benchmarking and runtime optimization

Model-side efficiency is only half the story. The other half is how the runtime executes the model.

10.1 Latency, throughput, concurrency, and percentiles

Suppose a service handles many requests.

Latency is the time for one request.

Throughput is requests per second or tokens per second.

Concurrency is how many requests are active at the same time.

Percentiles matter because average latency can hide bad tail behavior. A service with good mean latency but terrible 95th-percentile latency can still feel bad to users.

10.2 A simple mental experiment

Imagine an accelerator is underutilized when running one request. Adding more concurrent requests may increase throughput because idle hardware is used better. But if you keep increasing concurrency, requests start waiting for memory and compute, so latency rises. This is why a benchmark should report the operating point, not just one headline number.

10.3 Why prefill and decode should be measured separately

Prefill and decode stress the system differently.

- **Prefill** processes many prompt tokens at once, so arithmetic utilization can be high.
- **Decode** handles one or a few new tokens at a time while reading the full model and the KV cache repeatedly, so memory traffic often dominates.

A method that helps prefill may not help decode equally, and vice versa.

10.4 Three runtime ideas to keep in mind

FlashAttention. Standard attention can waste time moving large intermediate matrices in and out of high-bandwidth memory. FlashAttention reduces this traffic by tiling the computation and fusing the softmax logic [66, 67].

Speculative decoding. A cheaper draft model proposes several tokens; the larger target model verifies them. If many proposals are accepted, the target model needs fewer expensive decode steps [46, 47].

PagedAttention and continuous batching. LLM serving systems must manage the growing KV cache carefully. PagedAttention treats the KV cache like paged memory, which improves memory management and throughput under many concurrent requests [51].

10.5 A correct measurement protocol in one list

When benchmarking, always state:

- hardware and software stack;
- model version and precision;
- prompt length and generation length;
- batch size or concurrency;
- whether warmup was used;
- whether timings were synchronized correctly;
- what quality metric was checked after optimization.

The most common measurement fallacies

- Comparing different prompt lengths.
- Reporting only average latency.
- Ignoring warmup.
- Measuring GPU time without proper synchronization.
- Claiming speedup from lower FLOPs without reporting end-to-end latency.
- Reporting tokens/s from a different concurrency point than the latency number.

Checkpoint: the benchmarking rule

Question. What is the single most important rule for efficiency benchmarking?

Answer. Benchmark the actual deployed operating point and report the full context. The same model can look fast or slow depending on prompt length, output length, batch size, concurrency, kernel implementation, and hardware.

11 What to try first in practice

A common practical question is: “What should I try first?” The answer depends on the deployment constraint.

If you stop after Part I, what should you remember?

1. Quantization changes the *precision* of numbers.
2. Pruning changes the *structure* of the model.
3. Distillation changes the *training target* so a smaller model can imitate a larger one.
4. Low-rank methods change the *degrees of freedom* used to represent a matrix or update.
5. Benchmarking asks whether any of those changes actually improved the serving bottleneck.

If those five sentences feel clear, Part I has done its job.

Table 5: A simple deployment playbook.

Primary pain point	Usually good first moves	Why
Model does not fit in memory	Weight quantization; KV-cache quantization if needed	Reduces bytes immediately
Need faster GPU inference without retraining a whole new model	Good PTQ plus strong runtime kernels; FlashAttention-class runtime methods	Reduces memory traffic and improves execution efficiency
Need the best quality-latency frontier and re-training is allowed	Distill a smaller student; possibly quantize the student too	The small model is cheap by design
Need cheap task adaptation of a large model	LoRA or QLoRA; merge adapters when appropriate	Trains a small update rather than the full model
Need real speedups from sparsity	Prefer structured or hardware-supported semi-structured sparsity	Arbitrary zeros often do not accelerate

Part II

Rigorous survey and literature map

What Part II is for

Part II is intentionally denser than Part I. It develops the common optimization framework, gives full derivations, and traces the historical and recent paper lineage. It is meant to work well as a reference even if you do not read every line sequentially.

12 Notation and deployment view for the rigorous survey

We write a model as f_θ with parameters θ , a dataset as $\mathcal{D}$, inputs as x , labels or targets as y , and linear layers as $y = Wx$. The survey targets both classical deep networks and modern autoregressive language models, but the deployment logic is the same in both settings: quality must be preserved while model memory, online latency, offline throughput, and energy remain within budget.

Table 6: Symbols used repeatedly in the derivations.

Symbol	Meaning
$\theta, \tilde{\theta}$	Original and transformed model parameters
$W, \hat{W}$	Full-precision and modified weight matrices
X	Calibration or batch activation matrix whose columns are input examples
H	Hessian or a Hessian surrogate such as $XX^\top$
s, z	Quantization scale and zero-point
m	Binary pruning mask
r	Rank budget in low-rank approximation or low-rank adaptation
$T_{\text{prefill}}, T_{\text{decode}}$	Prompt-processing and generation-time latency components for LLMs

A useful first distinction is between *algorithmic compression* and *system realization*. Algorithmic compression changes the representation of the model or the training objective. System realization determines whether that change appears as an actual serving gain. For example, weight pruning can reduce parameter count without reducing latency if the runtime still dispatches dense kernels; conversely, a better KV-cache manager can improve throughput substantially without changing the model at all.

13 A unified mathematical framework for efficiency

Table 7: The main levers considered in this survey.

Lever	What changes	Strongest practical gain	Main deployment risk
Quantization	Numeric precision of weights, activations, or KV cache	Model size and memory bandwidth; sometimes kernel speed if low-bit kernels exist	Accuracy loss from outliers, clipping, poor calibration, or kernel fallback
Pruning / sparsity	Number or arrangement of nonzero parameters / sublayers	Speed only when sparsity pattern is hardware-friendly or dimensions actually shrink	FLOP reduction without end-to-end speedup
Distillation	Training objective and often the student architecture	Best quality-latency frontier when retraining a smaller student is allowed	Teacher bias and train-inference mismatch
Low-rank structure	Matrix factorization or low-rank updates	Cheap adaptation and parameter reduction with mergeable inference path	Too-small rank can bottleneck adaptation quality
Benchmarking	Measurement protocol and serving stack	Exposes the <i>true</i> bottleneck and prevents false claims	Apples-to-oranges comparisons and unsynchronized timing

13.1 Deployment-constrained optimization

The cleanest abstraction is to view efficiency as constrained empirical risk minimization on a transformed model. Let $\mathcal{T}$ denote a transformation such as quantization, pruning, rank factorization, or distillation-induced retraining. The deployed model is $\tilde{\theta} = \mathcal{T}(\theta)$. Let $M(\tilde{\theta})$ be model memory, $T_{p95}(\tilde{\theta}, \pi, \mathcal{H})$ be p95 latency under serving policy π on hardware $\mathcal{H}$, and $Q(\tilde{\theta}, \pi, \mathcal{H})$ be throughput. Then the deployment problem is

$$\begin{aligned}
& \min_{\tilde{\theta}, \pi} \mathcal{L}_{\text{task}}(\tilde{\theta}; \mathcal{D}_{\text{eval}}) \\
& \text{subject to} \quad M(\tilde{\theta}) \leq B_M, \\
& \quad T_{p95}(\tilde{\theta}, \pi, \mathcal{H}) \leq B_T, \\
& \quad Q(\tilde{\theta}, \pi, \mathcal{H}) \geq B_Q.
\end{aligned} \tag{1}$$

An equivalent Lagrangian form is

$$\min_{\tilde{\theta}, \pi} \mathcal{L}_{\text{task}}(\tilde{\theta}) + \lambda_M M(\tilde{\theta}) + \lambda_T T(\tilde{\theta}, \pi, \mathcal{H}) - \lambda_Q Q(\tilde{\theta}, \pi, \mathcal{H}). \tag{2}$$

Equation (1) is broad enough to include all topics in this survey:

- **Quantization** constrains $\tilde{\theta}$ to a discrete codebook.
- **Pruning** constrains $\tilde{\theta}$ to a sparse or lower-width topology.
- **Low-rank methods** constrain $\tilde{\theta}$ or $\Delta\theta$ to low rank.
- **Distillation** changes the training objective so that a smaller $\tilde{\theta}$ can solve the same task.
- **Benchmarking** estimates the functions M, T , and Q correctly.

13.2 Local quadratic view: why second-order structure keeps reappearing

Suppose the baseline model θ is already trained. A compression transform produces a perturbation δ so that $\tilde{\theta} = \theta + \delta$. Around θ , the task loss has the Taylor expansion

$$\mathcal{L}(\theta + \delta) = \mathcal{L}(\theta) + g^\top \delta + \frac{1}{2} \delta^\top H \delta + o(\|\delta\|^2), \quad (3)$$

where $g = \nabla \mathcal{L}(\theta)$ and $H = \nabla^2 \mathcal{L}(\theta)$ is the Hessian. Near a local optimum, $g \approx 0$, so the first nontrivial term is the quadratic penalty

$$\Delta \mathcal{L} \approx \frac{1}{2} \delta^\top H \delta. \quad (4)$$

This equation is the common mathematical root of many “post-training” methods. The transform only changes the feasible set:

$$\mathcal{C}_{\text{quant}} = \{\delta : \theta + \delta \in \mathcal{Q}_b\}, \quad \mathcal{C}_{\text{sparse}} = \{\delta : \|\theta + \delta\|_0 \leq k\}, \quad \mathcal{C}_{\text{rank}} = \{\delta : \text{rank}(\theta + \delta) \leq r\}.$$

The optimization becomes

$$\min_{\delta \in \mathcal{C}} \frac{1}{2} \delta^\top H \delta.$$

Quantization methods often approximate H by activation covariances; pruning methods use diagonal or inverse-Hessian surrogates; low-rank approximations solve the same idea under matrix-rank constraints.

13.3 Runtime lower bounds and the bottleneck principle

Algorithmic gains matter only if they improve the serving bottleneck. For one layer or kernel, let F be the required arithmetic operations and B be the moved bytes. If the effective compute rate is P_{eff} and the effective memory bandwidth is BW_{eff} , then

$$T_{\text{layer}} \gtrsim \max\left(\frac{F}{P_{\text{eff}}}, \frac{B}{BW_{\text{eff}}}\right) + T_{\text{launch}} + T_{\text{host}} + T_{\text{format}}, \quad (5)$$

where T_{format} collects packing, dequantization, synchronization, layout conversion, and similar

overheads. Define the arithmetic intensity $I = F/B$. If

$$I < \frac{P_{\text{eff}}}{BW_{\text{eff}}},$$

the layer is memory-bound. In that regime, reducing FLOPs alone may do very little. This is the central reason unstructured sparsity often underdelivers on GPUs, and the central reason low-bit quantization can help even when the arithmetic itself is already fast: it reduces bytes moved.

For autoregressive generation, request latency is more naturally decomposed as

$$T_{\text{req}} = T_{\text{queue}} + T_{\text{prefill}} + \sum_{t=1}^{L_{\text{out}}} T_{\text{decode},t} + T_{\text{post}}. \quad (6)$$

A method that speeds up prefill but not decode may help long prompts more than long outputs; a method that shrinks weights but not KV-cache traffic may help small-batch offline benchmarking but not online high-concurrency serving. The rest of the survey repeatedly returns to this bottleneck principle.

13.4 A short decision rule

An efficiency method is usually worth deploying only if it clears all three tests:

1. it preserves task quality on the relevant evaluation distribution,
2. it improves the bottleneck term in Equation (5) or Equation (6),
3. it does so under the same runtime, same batch/concurrency regime, and same accuracy target.

14 Quantization: basics, PTQ vs QAT, pitfalls, and recent work

14.1 First-pass picture: what quantization changes

There are three distinct quantization targets: weights, activations, and the KV cache. If a tensor with N scalar entries is stored in b bits, then its raw storage is $Nb/8$ bytes. Thus moving from FP16 (16 bits) to INT4 (4 bits) divides raw storage by four. That raw storage win does *not* by itself guarantee a $4\times$ latency speedup, because the runtime must still read scales, unpack bits, and invoke low-bit kernels.

For autoregressive models, KV-cache memory can be a dominant term. A simple scaling law is

$$M_{\text{KV}} \approx 2B n_{\text{layers}} n_{\text{heads}} d_{\text{head}} T \cdot \frac{b_{\text{KV}}}{8}, \quad (7)$$

where B is the number of active sequences, T is the cached context length, and the factor 2 accounts for storing both keys and values. Equation (7) is only a back-of-the-envelope model, but it explains why recent LLM work treats KV-cache quantization as a first-class deployment problem rather than an afterthought.

14.2 Quantization as constrained approximation

Consider a scalar $x \in \mathbb{R}$ that must be represented with b bits using an affine grid

$$\mathcal{C} = \{s(q - z) : q \in \{q_{\min}, \dots, q_{\max}\}\}, \quad q_{\max} - q_{\min} + 1 = 2^b,$$

where $s > 0$ is the scale and z is the zero-point. The scalar quantization problem is

$$\hat{x} = \arg \min_{\tilde{x} \in \mathcal{C}} (x - \tilde{x})^2. \quad (8)$$

Write $\tilde{x} = s(q - z)$. Then

$$(x - s(q - z))^2 = s^2 \left(\frac{x}{s} + z - q \right)^2.$$

Since s^2 is constant with respect to q , minimizing Equation (8) is equivalent to choosing the nearest integer code:

$$q^* = \text{clip} \left(\left\lfloor \frac{x}{s} \right\rfloor + z, q_{\min}, q_{\max} \right), \quad \hat{x} = s(q^* - z). \quad (9)$$

This is the basic affine quantizer used in integer inference stacks. Symmetric quantization is the special case $z = 0$.

14.3 How the scale and zero-point are obtained

Suppose we want the real interval $[a, b]$ to map to the integer interval $[q_{\min}, q_{\max}]$. The defining equations are

$$a = s(q_{\min} - z), \quad b = s(q_{\max} - z).$$

Subtracting the two equations gives

$$s = \frac{b - a}{q_{\max} - q_{\min}}. \quad (10)$$

Plugging back yields

$$z = q_{\min} - \frac{a}{s}. \quad (11)$$

In practice, z is rounded to an integer. For symmetric quantization with $a = -\beta$ and $b = \beta$, one sets

$$z = 0, \quad s = \frac{\beta}{q_{\max}}.$$

If clipping is negligible and x is approximately uniformly distributed within each quantization cell, then the quantization error $e = \hat{x} - x$ is approximately uniform on $[-s/2, s/2]$, so

$$\mathbb{E}[e] \approx 0, \quad \text{Var}(e) \approx \frac{s^2}{12}.$$

This simple variance estimate already explains a lot: smaller scales reduce noise, but overly small scales increase clipping. Calibration is the art of trading those two errors.

14.4 The historically important deployment bridge: integer-only inference

The affine formula in Equation (9) becomes deployable only when a layer can be evaluated mostly with integer arithmetic, the key bridge formalized by Jacob et al. [5]. For one dot product,

$$x_i \approx s_x(q_{x,i} - z_x), \quad w_i \approx s_w(q_{w,i} - z_w).$$

Then

$$y = \sum_{i=1}^d w_i x_i + b \tag{12}$$

$$\approx s_w s_x \sum_{i=1}^d (q_{w,i} - z_w)(q_{x,i} - z_x) + b. \tag{13}$$

Expand the product:

$$\sum_{i=1}^d (q_{w,i} - z_w)(q_{x,i} - z_x) = \sum_{i=1}^d q_{w,i} q_{x,i} - z_w \sum_{i=1}^d q_{x,i} - z_x \sum_{i=1}^d q_{w,i} + d z_w z_x. \tag{14}$$

All terms in Equation (14) are integer accumulations or precomputable corrections. If the output is also quantized as $y \approx s_y(q_y - z_y)$, then the final requantization step is

$$q_y \approx \text{clip} \left(\left\lfloor \frac{s_w s_x}{s_y} A + \frac{b}{s_y} \right\rfloor + z_y, q_{\min}, q_{\max} \right), \tag{15}$$

where A denotes the corrected integer accumulator from Equation (14). This is the core reason affine quantization became a deployment primitive rather than only a storage trick.

14.5 From scalar quantization to layer quantization

For a linear layer $y = Wx$, the quantized counterpart is $\hat{y} = \hat{W}\hat{x}$. The output error decomposes as

$$\hat{y} - y = (\hat{W} - W)x + \hat{W}(\hat{x} - x). \tag{16}$$

Thus weight and activation quantization errors enter asymmetrically. If decode is memory-bound, weight-only quantization may already help because it reduces weight traffic. If hardware supports fast low-bit matmuls, weight-and-activation quantization can do more. If activations contain outliers, however, the second term in Equation (16) dominates and naive post-training quantization fails.

14.6 PTQ vs QAT at a high level

PTQ solves a constrained approximation problem on a trained model. QAT instead changes the training dynamics so the model learns weights that are easier to quantize.

Table 8: High-level comparison of post-training quantization (PTQ) and quantization-aware training (QAT).

Aspect	PTQ	QAT
When applied	After full-precision training	During fine-tuning or retraining
Optimization signal	Calibration statistics, local reconstruction, or Hessian surrogates	End-to-end task loss with fake quantization in the loop
Compute cost	Low	Higher, often much higher
Typical strength	Fast deployment, especially for INT8 or weight-only INT4	Better recovery at aggressive bitwidths or difficult activation quantization
Main failure mode	Poor calibration, outliers, silent kernel mismatch	Optimization instability, overfitting, expensive training
When preferred	No retraining budget or need for quick model conversion	Accuracy drop from PTQ is unacceptable and training budget exists

14.7 Canonical PTQ derivation: from output reconstruction to GPTQ-style compensation

The most common PTQ objective for one linear layer is output reconstruction on a calibration set. Let $X \in \mathbb{R}^{d \times n}$ collect n input activation vectors, and let $W \in \mathbb{R}^{m \times d}$ be the full-precision weights. We want a quantized $\hat{W}$ minimizing the discrepancy

$$\arg \min_{\hat{W} \in \mathcal{Q}} \|WX - \hat{W}X\|_F^2. \quad (17)$$

Because the Frobenius norm is separable across rows, Equation (17) decomposes into

$$\arg \min_{\hat{w}_i \in \mathcal{Q}^d} \|w_i X - \hat{w}_i X\|_2^2, \quad i = 1, \dots, m.$$

Fix one row $w \in \mathbb{R}^d$ and write $\delta = \hat{w} - w$. Then

$$\|wX - \hat{w}X\|_2^2 = \|\delta X\|_2^2 = \delta(XX^\top)\delta^\top. \quad (18)$$

Define the Hessian surrogate

$$H = XX^\top.$$

Equation (18) becomes

$$\arg \min_{\hat{w} \in \mathcal{Q}^d} (\hat{w} - w)H(\hat{w} - w)^\top. \quad (19)$$

This is already the bridge from PTQ to second-order methods: activation covariance acts like a layer-local Hessian.

Now suppose we quantize one coordinate w_j to some codebook value q_j , but we are allowed to

compensate the remaining coordinates. Let $d = \hat{w} - w$ be a column vector and impose the constraint

$$e_j^\top d = q_j - w_j =: \alpha.$$

The constrained problem is

$$\min_d d^\top H d \quad \text{subject to} \quad e_j^\top d = \alpha.$$

Its Lagrangian is

$$\mathcal{J}(d, \lambda) = d^\top H d + \lambda(e_j^\top d - \alpha).$$

Stationarity gives

$$\nabla_d \mathcal{J} = 2Hd + \lambda e_j = 0 \quad \implies \quad d = -\frac{\lambda}{2} H^{-1} e_j.$$

Enforcing the constraint:

$$e_j^\top d = -\frac{\lambda}{2} e_j^\top H^{-1} e_j = -\frac{\lambda}{2} [H^{-1}]_{jj} = \alpha.$$

Hence

$$\lambda = -\frac{2\alpha}{[H^{-1}]_{jj}}, \quad d^* = \frac{\alpha}{[H^{-1}]_{jj}} H^{-1} e_j.$$

Since $\alpha = q_j - w_j$, the optimal compensation update can be written as

$$w \leftarrow w - \frac{w_j - q_j}{[H^{-1}]_{jj}} H_{:,j}^{-1}. \quad (20)$$

The minimum added reconstruction error is

$$(d^*)^\top H d^* = \frac{(q_j - w_j)^2}{[H^{-1}]_{jj}}. \quad (21)$$

Equations (20)–(21) are the core algebra behind Optimal Brain Surgeon style compensation and modern GPTQ-like row-wise PTQ [16, 9]. The final algorithm is:

1. Collect calibration activations X for each layer.
2. Build $H = XX^\top + \lambda I$ with damping λI for numerical stability.
3. Choose the quantization granularity: per-tensor, per-channel, or grouped.
4. For each row, and often for each block of columns, quantize one coordinate at a time to the nearest codebook value.
5. After fixing that coordinate, update the remaining unfixed coordinates using Equation (20).
6. Pack the resulting integer weights and re-run end-to-end evaluation.

This derivation also explains why grouped quantization often helps: smaller groups mean the local scale better matches the local distribution, which reduces quantization noise.

14.8 Canonical QAT derivation: fake quantization and the straight-through estimator

QAT solves

$$\min_{\theta, s} \mathbb{E}_{(x, y) \sim \mathcal{D}} \left[\ell(f_{\hat{\theta}(s)}(x), y) \right], \quad (22)$$

where $\hat{\theta}(s) = Q(\theta; s)$ is the quantized or fake-quantized parameterization. For one scalar variable v ,

$$\hat{v} = s \cdot \text{clip}\left(\left\lfloor \frac{v}{s} \right\rfloor, q_n, q_p\right). \quad (23)$$

The problem is that the rounding operator is non-differentiable and has zero derivative almost everywhere. QAT therefore uses the straight-through estimator (STE), replacing the true derivative by a surrogate.

Let

$$u = \frac{v}{s}, \quad q(u) = \text{clip}(\lfloor u \rfloor, q_n, q_p), \quad \hat{v} = sq(u).$$

Inside the unclipped region $q_n < u < q_p$, STE treats $dq/du \approx 1$. Therefore

$$\frac{\partial \hat{v}}{\partial v} = s \frac{\partial q}{\partial u} \frac{\partial u}{\partial v} \approx s \cdot 1 \cdot \frac{1}{s} = 1.$$

Outside the clipping region, the clip derivative is set to 0, so

$$\frac{\partial \hat{v}}{\partial v} \approx \mathbf{1}_{\{q_n < v/s < q_p\}}. \quad (24)$$

If the scale s is learned, then

$$\frac{\partial \hat{v}}{\partial s} = q(u) + s \frac{\partial q}{\partial u} \frac{\partial (v/s)}{\partial s}.$$

Inside the unclipped region, $\partial q/\partial u \approx 1$ and $\partial(v/s)/\partial s = -v/s^2 = -u/s$, giving

$$\frac{\partial \hat{v}}{\partial s} \approx q(u) - u = \left\lfloor \frac{v}{s} \right\rfloor - \frac{v}{s}. \quad (25)$$

If $u \leq q_n$, the clip derivative vanishes and

$$\frac{\partial \hat{v}}{\partial s} \approx q_n.$$

If $u \geq q_p$,

$$\frac{\partial \hat{v}}{\partial s} \approx q_p.$$

These are exactly the LSQ-style surrogate gradients used to learn quantization scales [54].

The canonical QAT recipe is therefore:

1. Keep latent full-precision weights.
2. Insert fake-quant modules on weights and usually activations.

3. Run the forward pass with Equation (23).
4. Backpropagate with surrogate gradients such as Equations (24)–(25).
5. Optionally learn clipping thresholds or scales.
6. Export the final graph to the exact integer or low-bit kernel stack intended for deployment.

QAT tends to outperform PTQ at low bitwidth because the model is optimized *for* the quantized forward pass instead of merely being projected onto a codebook after training.

14.9 Historical derivation snapshot: PACT and learnable clipping

PACT [6] is historically important because it made the clipping threshold itself trainable. For a nonnegative activation a (for example after ReLU), PACT writes the clipped activation as

$$\tilde{a} = \frac{1}{2}(|a| - |a - \alpha| + \alpha) = \text{clip}(a, 0, \alpha), \quad \alpha > 0. \quad (26)$$

The quantizer then maps $\tilde{a}$ to a k -bit grid with scale

$$s_a = \frac{\alpha}{2^k - 1}.$$

The key point is that α is optimized by gradient descent together with the network weights. The piecewise derivative is easy to see from the clipping form:

$$\frac{\partial \tilde{a}}{\partial \alpha} = \begin{cases} 0, & a < \alpha, \\ 1, & a > \alpha, \end{cases}$$

ignoring the single nondifferentiable point $a = \alpha$. So if too many activations saturate at the clip ceiling, the loss pushes α upward; if the dynamic range is unnecessarily wide, optimization can shrink α and reduce quantization noise. PACT therefore provided an early, very concrete lesson that later LSQ-style methods made standard: the quantizer parameters are part of the model and should often be learned, not fixed by hand.

Plain-language interpretation. A fixed clipping threshold is a guess. PACT turns that guess into a trainable parameter so the network can decide how much dynamic range it truly needs.

14.10 Historical derivation snapshot: why Hessian trace drives mixed precision

The most influential historical explanation for mixed-precision quantization is the HAWQ line [55, 56]. Consider one layer l with quantization perturbation δ_l . A local quadratic approximation gives

$$\Delta \mathcal{L}_l \approx \frac{1}{2} \delta_l^\top H_l \delta_l.$$

If the quantization noise is approximately zero-mean with covariance $\Sigma_l = \mathbb{E}[\delta_l \delta_l^\top]$, then

$$\mathbb{E}[\Delta \mathcal{L}_l] \approx \frac{1}{2} \text{tr}(H_l \Sigma_l). \quad (27)$$

If a coarse approximation models the quantization noise as isotropic, $\Sigma_l \approx \sigma_l^2 I$, then

$$\mathbb{E}[\Delta \mathcal{L}_l] \approx \frac{1}{2} \sigma_l^2 \text{tr}(H_l).$$

This gives the central HAWQ intuition: layers with larger Hessian trace are more error-sensitive and should receive more bits.

A simplified mixed-precision allocation problem is therefore

$$\min_{b_1, \dots, b_L} \sum_{l=1}^L \frac{1}{2} \text{tr}(H_l \Sigma_l(b_l)) \quad \text{subject to} \quad \sum_{l=1}^L c_l b_l \leq B_{\text{bits}}, \quad (28)$$

where b_l is the bitwidth of layer l and c_l is its size. HAWQ introduced the second-order sensitivity view [55]; HAWQ-V2 argued that trace is a better summary than just the top eigenvalue and used this idea to search mixed-precision Pareto frontiers more systematically [56]. Equation (27) also explains why later PTQ methods keep reusing activation covariance or Hessian surrogates: they are trying to estimate the same curvature-weighted reconstruction damage.

14.11 Historical derivation snapshot: AdaRound and BRECQ

Naive PTQ uses round-to-nearest. AdaRound [8] asked a better question: if the goal is to preserve the *layer output*, why should rounding be fixed coordinatewise? For one scalar weight w with scale s , write

$$\frac{w}{s} = \lfloor w/s \rfloor + \rho, \quad \rho \in [0, 1).$$

Round-to-nearest chooses the extra bit

$$b = \begin{cases} 1, & \rho \geq 1/2, \\ 0, & \rho < 1/2. \end{cases}$$

AdaRound relaxes this hard binary decision by introducing a learnable soft rounding variable $h(v) \in [0, 1]$:

$$\hat{w}(v) = s(\lfloor w/s \rfloor + h(v)). \quad (29)$$

The optimization objective is a reconstruction loss plus a regularizer that gradually pushes $h(v)$ toward $\{0, 1\}$:

$$\min_v \left\| WX - \hat{W}(v)X \right\|_F^2 + \lambda R(v). \quad (30)$$

This is historically important because it recasts rounding itself as an optimization variable, rather than a hard preprocessing step.

BRECQ [57] extends the same idea from a single layer or row to *blocks*. Let f_B denote one residual

block and let $Y_B = f_B(X_B; W_B)$ be its floating-point output on calibration inputs X_B . BRECCQ solves

$$\min_{\hat{W}_B} \|Y_B - f_B(X_B; \hat{W}_B)\|_F^2, \quad (31)$$

often with second-order sensitivity estimates used to select mixed precision and to justify the blockwise decomposition. The central practical lesson is that reconstructing whole blocks gives a better compromise between

1. **locality** (small enough problems to optimize cheaply), and
2. **cross-layer coupling** (large enough problems to account for interactions inside a residual block).

Plain-language interpretation. AdaRound says “do not trust hard rounding blindly.” BRECCQ says “and do not optimize weights one scalar at a time when a residual block is the natural functional unit.”

14.12 Numerical pitfalls in quantization

The most common practical failures are not conceptual; they are numerical and systems-level.

Calibration mismatch. Calibration data that is too small, too clean, or too short-context underestimates activation range. This is especially harmful for autoregressive models because long-context and decode-time activations often differ from prefill activations.

Outliers and heavy tails. Activations can have a few channels with much larger magnitudes than the rest. Min-max scaling then allocates too much dynamic range to those channels, inflating the quantization step for everybody else.

Granularity mismatch. Per-tensor scales are cheapest, but per-channel or grouped scales better match heterogeneous weight distributions. The best choice is hardware-dependent.

Accumulator overflow and requantization. INT8 matmuls usually accumulate in INT32. Incorrect scaling, bias fusion, or requantization can silently overflow or saturate.

Silent kernel fallback. A model may be “quantized” in storage but dequantized back to FP16 for computation because the runtime lacks a supported kernel. This reduces memory but not compute, and often not latency.

Format overhead. Bit packing, dequantization, and layout conversion can erase arithmetic gains. This is especially common for exotic bitwidths and poorly integrated custom kernels.

Evaluation on the wrong metric. Small perplexity increases can correspond to larger instruction-following or reasoning drops. Layer reconstruction error alone is not enough.

14.13 How to evaluate accuracy drop correctly

A disciplined quantization evaluation should report at least the following:

1. the exact model, bitwidth, scale granularity, group size, and calibration protocol,
2. the *same* evaluation suite before and after quantization,
3. end-to-end latency or throughput on the *same* serving stack,
4. memory footprint, and ideally energy,
5. whether kernels are true low-bit kernels or dequantize-to-high-precision fallbacks.

For generative models, also separate prefill and decode, because the bottleneck often shifts between them. Report prompt length and generated length explicitly. For online systems, p50 and p95 are more informative than the mean.

14.14 Historical lineage

The quantization story has several identifiable roots. First, BNN/QNN asked whether networks could be trained with discrete weights or activations at all [3, 4]. Second, Jacob et al. made affine integer-only inference practical in production [5]. Third, PACT and LSQ made the QAT quantizer itself trainable through learnable clipping or step sizes [6, 54]. Fourth, HAWQ/HAWQ-V2 and later AdaRound/BRECQ made PTQ more optimization-aware by injecting curvature, learned rounding, and block reconstruction [55, 56, 8, 57]. Finally, the LLM era added outlier handling, scalable Hessian approximations, and system-aware implementations through LLM.int8(), ZeroQuant, GPTQ, and their descendants [10, 58, 9, 59].

Table 9: Quantization lineage from roots to current LLM practice.

Work	Year	Lasting idea	Why it still matters
Jacob et al.	2018	Affine scale/zero-point plus requantization	Still the deployment backbone of integer inference stacks
PACT	2018	Learn the clipping range	Early evidence that the quantizer itself should be optimized
LSQ	2019–20	Learn step sizes with surrogate gradients	Canonical QAT recipe for trainable scales
HAWQ / HAWQ-V2	2019–20	Use Hessian information for mixed precision	Established second-order sensitivity as a serious design principle
AdaRound	2020	Optimize rounding rather than rounding naively	Turned PTQ into a real optimization problem
BRECQ	2021	Blockwise reconstruction with second-order analysis	Strong bridge from vision PTQ to later large-model PTQ
ZeroQuant / V2	2022–23	LLM PTQ pipeline plus low-rank compensation	Showed that transformer PTQ must be co-designed with kernels and sometimes compensation
GPTQ / LLM.int8()	2022	Scalable rowwise PTQ; explicit outlier handling	Main bridge to practical LLM weight-only quantization

14.15 Recent influential works and how they extend the canonical PTQ/QAT framework

14.15.1 LLM.int8(): mixed-precision decomposition for outlier dimensions

LLM.int8() [10] is a pivotal practical bridge between classical INT8 quantization and modern LLM deployment. Its starting observation is that most hidden dimensions of transformer activations are well behaved, but a small set of *outlier* dimensions can dominate the error. Let the input activation be decomposed as

$$x = x_{\text{main}} + x_{\text{out}},$$

where x_{out} is nonzero only on a small set of outlier coordinates. Then a linear layer can be evaluated as

$$Wx = Wx_{\text{main}} + Wx_{\text{out}}. \quad (32)$$

LLM.int8() computes the first term in INT8 and the second term in higher precision. Algebraically this is trivial; deployment-wise it is powerful because more than 99.9% of multiplies can still happen in low precision while the few catastrophic outlier interactions are protected in FP16/BF16. Relation to the common framework: this is still a PTQ method, but it augments the feasible set with a tiny mixed-precision escape hatch for sensitive coordinates. The paper substantiates the idea by showing near-lossless 8-bit inference for very large transformers and by open-sourcing a software path that made such models much easier to serve in practice [10].

14.15.2 GPTQ: scalable row-wise second-order PTQ

GPTQ [9] is the most influential direct descendant of the row-wise reconstruction derivation in Equation (20). The paper’s contribution is not a different objective, but a scalable implementation of that objective for billion-parameter transformers: damped Hessian surrogates, blockwise inverse updates, and weight packing suited to deployment. Relative to the canonical framework, GPTQ is the baseline case where the theory and the final algorithm almost coincide exactly. The empirical substantiation is strong because the method reports near-lossless 3–4 bit weight-only quantization on large transformer families with calibration-only optimization [9].

14.15.3 SmoothQuant: diagonal rebalancing of activation and weight difficulty

SmoothQuant [11] is a historically important bridge from classical PTQ to modern LLM quantization. Its key observation is that for a diagonal matrix $S \succ 0$,

$$XW = (XS^{-1})(SW). \quad (33)$$

Equation (33) is mathematically exact. Therefore one can shrink activation outliers by dividing activation channels by S , while multiplying the corresponding weight channels by S to keep the layer output unchanged.

A natural balancing condition is to equalize scaled activation and weight maxima per channel:

$$\frac{\max |X_j|}{s_j} \approx s_j \max |W_j| \implies s_j^2 \approx \frac{\max |X_j|}{\max |W_j|}.$$

This gives the symmetric balance point

$$s_j \approx \left(\frac{\max |X_j|}{\max |W_j|} \right)^{1/2}.$$

SmoothQuant generalizes this with a migration-strength parameter α :

$$s_j = \frac{\max |X_j|^\alpha}{\max |W_j|^{1-\alpha}}.$$

Relation to the canonical framework: SmoothQuant modifies the feasible set before solving the PTQ problem in Equation (17). It does not change the architecture or require backpropagation; it changes the conditioning of the quantization problem. The paper’s main claim is that this exact reparameterization enables W8A8 quantization for all matrix multiplications in large language models with negligible accuracy loss and real inference benefits [11].

14.15.4 AWQ: activation-aware protection of salient channels

AWQ [12] starts from weight-only PTQ but asks a sharper question: which weight channels matter most? Its answer is activation-aware saliency. Empirically, protecting the channels associated with large-magnitude activations is much more effective than selecting channels by weight magnitude alone.

The same output-preserving scaling identity underlies AWQ. For a diagonal scaling S ,

$$Wx = (WS)(S^{-1}x).$$

Suppose a salient channel is scaled up. The quantization step is determined at the group level, but the effective *relative* error on the salient channel shrinks, while the input is inversely scaled to preserve the floating-point output. AWQ therefore chooses per-channel scales by minimizing the output mismatch after quantization:

$$\arg \min_S \left\| WX - Q(WS)(S^{-1}X) \right\|_F^2, \quad (34)$$

with a simple activation-dependent search space for S and optional clipping. Relation to the common framework: AWQ is still solving Equation (17), but it enriches the parameterization of the quantizer with a learned preconditioning transform. It improves over plain round-to-nearest and often over heavier reconstruction methods by focusing model capacity where the calibration activations indicate it matters most [12].

14.15.5 OmniQuant: differentiable PTQ with learnable clipping and equivalent transformations

OmniQuant [60] starts from the same block-reconstruction view as BRECQ, but re-optimizes the quantizer parameters themselves on each transformer block. In simplified form, for a block B with

inputs X_B and outputs $Y_B = f_B(X_B; W_B)$, the blockwise objective is

$$\min_{\phi, \psi} \|Y_B - f_B(X_B; Q_W(W_B; \phi), Q_A(\psi(X_B)))\|_F^2, \quad (35)$$

where ϕ collects learnable clipping thresholds and ψ collects output-preserving equivalent transformations.

A simple learnable weight-clipping model is

$$W_B^{\text{clip}} = \text{clip}(W_B, -c, c), \quad c > 0 \text{ learned},$$

followed by quantization $Q_W(W_B^{\text{clip}}; \phi)$. On the activation side, OmniQuant generalizes SmoothQuant- or AWQ-style channel scaling into a differentiable equivalent transformation layer.

Relation to the common framework: Equation (35) is still the reconstruction problem of Equation (17), but with a richer differentiable parameterization of the quantizer. The key claim is that calibration-time optimization of clipping and equivalent transforms makes low-bit PTQ much stronger, especially for hard settings such as W4A4 and W3A16 [60].

14.15.6 LLM-QAT: canonical QAT plus data-free self-distillation and KV-cache quantization

LLM-QAT [61] answers a simple question: when PTQ breaks at 4 bits and below, can we recover accuracy without the original training corpus? Its answer is to run vanilla fake-quant QAT, but on teacher-generated data and with a distillation loss from the full-precision model. A simplified objective is

$$\min_{\theta_q} \mathbb{E}_{x, \tilde{y} \sim P_{\theta_{\text{fp}}}} \left[\sum_t \text{KL} \left(p_{\theta_{\text{fp}}}(\cdot | x, \tilde{y}_{<t}) \parallel p_{\theta_q}^{\text{fake-quant}}(\cdot | x, \tilde{y}_{<t}) \right) \right]. \quad (36)$$

Equation (36) is just Equation (54) plugged into the canonical QAT pipeline, but with self-generated trajectories instead of the original supervised dataset. The paper also quantizes the KV cache, which matters because the KV cache can dominate memory in long-context generation. Relation to the common framework: LLM-QAT is not a new quantizer; it is the standard QAT objective made practical for LLM deployment. Its main claim is that QAT materially outperforms training-free PTQ in low-bit settings where PTQ starts to collapse [61].

14.15.7 BitDistiller: confidence-aware distillation for sub-4-bit QAT

BitDistiller [62] keeps the LLM-QAT recipe but modifies the distillation loss. Forward KL, $\text{KL}(P_T \| P_S)$, is mode-covering; reverse KL, $\text{KL}(P_S \| P_T)$, is mode-seeking. BitDistiller interpolates between them using teacher confidence:

$$D_{\text{CAKLD}}(P_T \| P_S) = \gamma \text{KL}(P_S \| P_T) + (1 - \gamma) \text{KL}(P_T \| P_S), \quad (37)$$

with

$$\gamma = \mathbb{E}_{(x,y) \sim \mathcal{D}} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} P_T(y_t \mid x, y_{<t}) \right]. \quad (38)$$

When the teacher is confident, γ is larger and the objective becomes more mode-seeking; when the teacher is uncertain, the objective becomes more mode-covering. Relation to the common framework: BitDistiller is canonical QAT plus a smarter distillation divergence. The paper’s main claim is that this simple change makes 2-bit and 3-bit QAT substantially stronger than earlier PTQ and QAT baselines, especially on reasoning-heavy tasks [62].

14.15.8 EfficientQAT: reducing the optimization cost barrier of LLM QAT

A practical objection to QAT is that it is expensive. EfficientQAT [63] addresses exactly that bottleneck by splitting the optimization into two phases: a block-wise phase that updates model parameters under fake quantization, followed by a lighter end-to-end phase that focuses on quantization parameters such as step sizes. In a simplified notation, one may view the method as approximating

$$\min_{\theta, s} \mathcal{L}_{\text{task}}(f_{Q(\theta; s)})$$

through alternating subproblems

$$\begin{aligned} \theta &\leftarrow \arg \min_{\theta} \mathcal{L}_{\text{task}}(f_{Q(\theta; s)}) \quad \text{on one block at a time,} \\ s &\leftarrow \arg \min_s \mathcal{L}_{\text{task}}(f_{Q(\theta; s)}) \quad \text{end-to-end.} \end{aligned}$$

Relation to the canonical framework: the objective is still standard QAT, but the optimization schedule is redesigned to make very low-bit QAT feasible at LLM scale. The paper substantiates this claim with low-bit results up to very large models while reporting a materially lower training cost than naive end-to-end QAT [63].

Plain-language interpretation. LLM-QAT showed that QAT can rescue very low bitwidths; EfficientQAT makes that rescue operation much cheaper to carry out.

14.15.9 QuaRot: orthogonal rotations to remove outliers

QuaRot [13] extends the reparameterization idea from diagonal scaling to orthogonal rotations. Let Q be orthogonal, so $Q^\top Q = I$. Then for a hidden representation h and a linear map W ,

$$hW = (hQ^\top)(QW).$$

By inserting such identities across the network and correspondingly rotating the weights, one can redistribute outliers across coordinates without changing the full-precision computation. The same idea applies to residual streams and, with the right algebra, to RMSNorm and KV-cache pathways.

The key principle is norm preservation:

$$\|hQ\|_2 = \|h\|_2.$$

Because the energy is spread more evenly across coordinates after rotation, uniform low-bit quantization becomes easier. Relation to the canonical framework: QuaRot changes the coordinate system in which Equation (17) is solved, making the quantization feasible set much better aligned with the data. The paper’s headline claim is end-to-end 4-bit quantization of weights, activations, and KV cache with around 99% retention of zero-shot performance [13].

14.15.10 SpinQuant: learned rotations instead of random ones

QuaRot shows that rotations help; SpinQuant [14] shows that the *choice* of rotation matters. The paper formulates the problem as learning orthogonal transformations on a calibration set:

$$\min_{Q \in \mathcal{O}(d)} \sum_{x \in \mathcal{C}} \|f_\theta(x) - f_{\theta, Q}^{\text{quant}}(x)\|_2^2, \quad (39)$$

where $\mathcal{O}(d)$ is the orthogonal group. Orthogonality is maintained with Cayley-style updates. If $A^\top = -A$ is skew-symmetric, then the Cayley transform

$$Q(A) = \left(I - \frac{1}{2}A\right)^{-1} \left(I + \frac{1}{2}A\right) \quad (40)$$

is orthogonal whenever the inverse exists. This gives a differentiable parameterization of rotations. Relation to the canonical framework: SpinQuant optimizes the coordinate system to minimize the same post-quantization discrepancy as Equation (17), but unlike QuaRot it learns that system instead of selecting a fixed Hadamard-style transform. The paper reports that with 4-bit quantization of weights, activations, and KV cache, the gap to full precision on zero-shot reasoning can be reduced to a few points and that learned rotations outperform both random rotations and strong baselines such as SmoothQuant and LLM-QAT [14].

Checkpoint. PTQ changes a trained model after the fact; QAT changes the training loop so the model learns to live with low precision. In practice, PTQ is the fast deployment tool and QAT is the rescue tool when PTQ loses too much accuracy.

15 Pruning and sparsity: unstructured vs structured, and when speedups are real

15.1 Pruning as constrained optimization

For pruning, the canonical constrained problem is

$$\min_{m, W^c} \mathcal{L}(m \odot W^c) \quad \text{subject to} \quad \|m\|_0 \leq k, \quad (41)$$

Table 10: Recent influential quantization works and how they relate to the canonical PTQ/QAT framework.

Work	Year	How it fits the common framework	What changed in practice
LLM.int8()	2022	PTQ with an explicit mixed-precision decomposition for outlier dimensions	Made large-transformer 8-bit inference practical without noticeable quality loss in many cases
GPTQ	2022	Direct scalable implementation of row-wise second-order PTQ	Became a standard weight-only low-bit baseline for LLMs
SmoothQuant	2022–23	Exact diagonal reparameterization before PTQ; migrates difficulty from activations to weights	Made W8A8 PTQ credible for mainstream LLM inference
AWQ	2023	Weight-only PTQ with activation-aware scaling and output reconstruction	Strong 4-bit accuracy with relatively light calibration; influential in practical toolchains
OmniQuant	2023–24	PTQ with learnable clipping and differentiable equivalent transformations	Strengthened hard settings such as W4A4 by optimizing quantizer parameters directly
LLM-QAT	2023–24	Standard fake-quant QAT plus data-free self-distillation and KV-cache quantization	Showed that QAT can recover low-bit settings where PTQ starts to fail
BitDistiller / EfficientQAT	2024	Same QAT objective, but with stronger distillation or cheaper optimization schedules	Pushed sub-4-bit QAT toward practical feasibility
QuaRot / SpinQuant	2024	Orthogonal preconditioning before PTQ, fixed in QuaRot and learned in SpinQuant	Made end-to-end 4-bit W/A/KV quantization much more realistic

where $m \in \{0, 1\}^{\dim(W)}$ is a binary mask and W^c are the compensated or retrained values of the surviving weights.

Unstructured pruning treats m elementwise. Structured pruning defines groups $G_1, \dots, G_G$ (channels, heads, rows, columns, blocks, or whole sublayers) and removes groups rather than individual entries. Semi-structured patterns such as $N:M$ sparsity sit between the two.

15.2 Detailed derivation of the classical unstructured framework: OBD, magnitude pruning, and OBS

Let θ be a trained parameter vector and consider removing a single coordinate θ_i . Using Equation (3) near a local optimum, $g \approx 0$, so

$$\Delta\mathcal{L} \approx \frac{1}{2}\delta^\top H\delta.$$

Optimal Brain Damage (OBD). If we remove coordinate i and do *not* compensate the others, then

$$\delta = -\theta_i e_i.$$

Therefore

$$\Delta\mathcal{L}_i \approx \frac{1}{2}H_{ii}\theta_i^2. \quad (42)$$

This is the OBD saliency [15]. If the diagonal curvatures H_{ii} are treated as approximately constant, the ranking reduces to magnitude pruning:

$$\Delta\mathcal{L}_i \propto \theta_i^2.$$

That is the mathematical reason magnitude pruning is often a surprisingly strong baseline.

Optimal Brain Surgeon (OBS). If we *do* allow compensation in the remaining coordinates while forcing coordinate i to zero, the problem becomes

$$\min_{\delta} \frac{1}{2}\delta^\top H\delta \quad \text{subject to} \quad e_i^\top \delta + \theta_i = 0.$$

The Lagrangian is

$$\mathcal{J}(\delta, \lambda) = \frac{1}{2}\delta^\top H\delta + \lambda(e_i^\top \delta + \theta_i).$$

Stationarity yields

$$H\delta + \lambda e_i = 0 \quad \implies \quad \delta = -\lambda H^{-1}e_i.$$

Applying the constraint:

$$e_i^\top \delta + \theta_i = 0 \quad \implies \quad -\lambda[H^{-1}]_{ii} + \theta_i = 0,$$

so

$$\lambda = \frac{\theta_i}{[H^{-1}]_{ii}}.$$

Therefore the optimal compensation update is

$$\delta_i^* = -\frac{\theta_i}{[H^{-1}]_{ii}}H_{:,i}^{-1}, \quad (43)$$

and the corresponding loss increase is

$$\Delta\mathcal{L}_i^* = \frac{1}{2} \frac{\theta_i^2}{[H^{-1}]_{ii}}. \quad (44)$$

Equations (43)–(44) are the exact algebraic core of second-order one-shot pruning [16]. Modern LLM pruning methods such as SparseGPT inherit this structure, but replace the full model Hessian by layerwise activation-covariance surrogates.

15.3 Canonical structured pruning framework: group sparsity and its proximal step

For structured pruning, a standard formulation is group regularization:

$$\min_W \mathcal{L}(W) + \lambda \sum_{g=1}^G \|W_{G_g}\|_2. \quad (45)$$

The ℓ_2 norm inside the sum preserves within-group dense structure; the sum across groups drives entire groups to zero. The proximal gradient update for one group is

$$V_g = W_{G_g} - \eta \nabla_{G_g} \mathcal{L}(W),$$

followed by the proximal map

$$W_{G_g}^+ = \arg \min_u \frac{1}{2} \|u - V_g\|_2^2 + \eta \lambda \|u\|_2. \quad (46)$$

To solve Equation (46), note that the optimum must align with V_g , so set $u = \alpha V_g$ with $\alpha \geq 0$. Then

$$0 \in u - V_g + \eta \lambda \partial \|u\|_2.$$

For $u \neq 0$, $\partial \|u\|_2 = u / \|u\|_2$, so

$$\alpha V_g - V_g + \eta \lambda \frac{V_g}{\|V_g\|_2} = 0.$$

This simplifies to

$$(\alpha - 1) \|V_g\|_2 + \eta \lambda = 0 \implies \alpha = 1 - \frac{\eta \lambda}{\|V_g\|_2}.$$

If this is negative, the optimum is $u = 0$. Therefore

$$W_{G_g}^+ = \left(1 - \frac{\eta \lambda}{\|V_g\|_2}\right)_+ V_g. \quad (47)$$

Equation (47) is the group-wise analogue of soft thresholding and explains why group lasso naturally yields channel-, head-, or block-level pruning.

15.4 Historical derivation snapshot: network slimming and channel saliency

Network Slimming [20] is historically important because it turned structured pruning into a simple training-time recipe. In a batch-normalized network, each channel has a scale parameter γ_c . If one adds an ℓ_1 penalty,

$$\min_{\theta, \gamma} \mathcal{L}(\theta, \gamma) + \lambda \sum_c |\gamma_c|, \quad (48)$$

then channels with small utility are encouraged to have small γ_c . After training, channels with γ_c near zero can be removed entirely.

Why does this work mathematically? The proximal update for one scale parameter is the scalar

soft-thresholding rule

$$\gamma_c^+ = \text{sign}(v_c) \max(|v_c| - \eta\lambda, 0),$$

where v_c is the gradient step before the proximal shrinkage. Thus the optimization itself drives many channel scales exactly to zero. This is the structured analogue of lasso sparsity, but on channel gates rather than individual weights.

Plain-language interpretation. Instead of asking after training which channels to delete, network slimming trains the network so that useless channels reveal themselves through nearly zero scale parameters.

15.5 Unstructured vs structured vs semi-structured sparsity

Unstructured sparsity removes individual weights. It offers the most flexibility and usually the best compression at a given sparsity level, but it rarely maps cleanly to commodity dense kernels.

Structured sparsity removes rows, columns, channels, heads, MLP neurons, attention heads, or entire sublayers. It reduces tensor shapes directly. This almost always gives the cleanest path to actual speedup because the resulting model remains dense, just smaller.

Semi-structured sparsity imposes local constraints such as 2:4 or 4:8: in every block of M weights, only N may remain nonzero. This balances hardware regularity with pruning flexibility. On hardware with dedicated sparse tensor cores, it can yield real speedups.

15.6 When sparsity yields real speedups

The correct answer is: only when the runtime and hardware can exploit the pattern *and* the removed work lies on the bottleneck path.

For a dense GEMM with shapes $(m \times k)$ and $(k \times n)$,

$$F_{\text{dense}} = 2mkn.$$

If unstructured pruning leaves a density ρ , a *theoretical* sparse arithmetic count is

$$F_{\text{sparse}} \approx 2\rho mkn.$$

But the end-to-end time is not determined by arithmetic count alone. A more realistic sparse lower bound is

$$T_{\text{sparse}} \gtrsim \max\left(\frac{2\rho mkn}{P_{\text{sparse}}}, \frac{B_x + \rho B_w + B_{\text{idx}} + B_y}{BW_{\text{sparse}}}\right).$$

If the implementation actually routes through dense kernels, then P_{sparse} and BW_{sparse} are effectively the dense values and the tensor shapes remain unchanged. In that case, the model may be smaller on disk but not faster in memory or latency.

By contrast, if structured pruning removes a fraction r of output channels, then $n' = (1 - r)n$, so

both compute and weight traffic shrink:

$$F' = 2mk(1 - r)n, \quad B'_w \propto k(1 - r)n.$$

Now the dense kernel itself operates on smaller matrices. This is why structured pruning almost always translates into more reliable speedups than unstructured pruning.

A practical summary:

- **Unstructured sparsity** is mainly useful for research comparisons, compressed storage, or specialized sparse runtimes.
- $N:M$ **sparsity** is the most credible route to GPU speedups when hardware support exists.
- **Structured pruning** is the safest route to real latency reduction because the model dimensions themselves shrink.

15.7 Historical lineage

Pruning begins with OBD and OBS [15, 16]. The deep learning era added magnitude-based iterative pruning and the full deep compression pipeline [17, 18]. Structured pruning then focused on filter, channel, and slimming methods [19, 20]. Two historically important developments broadened the picture further: the Lottery Ticket Hypothesis emphasized that sparse subnetworks can be trainable in their own right, and movement pruning showed that during fine-tuning one should often prune weights whose update *trajectory* moves them toward zero rather than weights that merely have small current magnitude [21, 22]. A critical later realization, emphasized by Gale et al., is that sparsity quality depends strongly on optimization and training protocol, not only on pruning criterion [23]. In parallel, hardware work made regular patterns such as 2:4 relevant for real acceleration [24]. In modern LLMs, the field split into one-shot unstructured pruning (SparseGPT), cheap saliency scores (Wanda), coupled structural pruning with short recovery tuning (LLM-Pruner), retraining-free structured pruning (FLAP), and smaller-dense-model approaches such as SliceGPT [25, 26, 64, 65, 27].

15.8 Recent influential works and their relation to the classical framework

15.8.1 SparseGPT: one-shot second-order pruning for LLMs

SparseGPT [25] applies the OBS logic layerwise to huge language models. For one layer with inputs X and weights W , the objective is

$$\arg \min_{M, W^c} \|WX - (M \odot W^c)X\|_F^2. \quad (49)$$

As in PTQ, this decomposes by rows. For a fixed row, the problem is quadratic in the row perturbation with Hessian surrogate $H = XX^\top$. Therefore the OBS update derived in Equation (43) is exact for the rowwise reconstruction objective. SparseGPT’s contribution is to engineer this computation so it scales to giant models: blockwise processing, shared Hessian approximations, and numerically

Table 11: Historical pruning milestones and the idea they contributed.

Work	Year	Key idea	Why it still matters
OBD / OBS	1990/93	Local quadratic saliency; optionally compensate surviving weights	Still the clearest mathematical root of pruning criteria
Han et al. / Deep Compression	2015–16	Iterative magnitude pruning plus quantization/coding pipeline	Established pruning as a practical compression primitive
Filter pruning / Network Slimming	2017	Remove hardware-friendly structures rather than isolated weights	Bridge from compression to real speedups
Lottery Ticket / Movement Pruning	2019–20	Sparse subnetworks can train well; fine-tuning trajectories matter	Important reminder that pruning is also an optimization problem
State of Sparsity	2019	Training protocol matters as much as pruning metric	Warning against over-attributing gains to a saliency formula alone
$N : M$ sparse acceleration	2021	Semi-structured patterns designed for hardware support	Still the most credible GPU sparsity path when supported

stable inverse/Hessian factorization. Relation to the common framework: SparseGPT is the pruning analogue of GPTQ. It does not change the architecture or training objective; it solves the local quadratic projection induced by Equation (4) at layer scale.

15.8.2 Wanda: activation-weight product as a diagonal-Hessian saliency

Wanda [26] is influential because it is much simpler than SparseGPT while remaining competitive. Its score for weight w_{ij} in row i , column j is

$$s_{ij} = |w_{ij}| \|x_j\|_2, \quad (50)$$

where x_j is the calibration activation along input channel j .

This score falls directly out of the OBD-style quadratic approximation under a diagonal Hessian surrogate. For rowwise reconstruction,

$$\Delta \approx d^\top H d, \quad H \approx \text{diag}(h_1, \dots, h_d), \quad h_j \propto \|x_j\|_2^2.$$

If weight w_{ij} is zeroed without compensation, the cost is approximately

$$\Delta_{ij} \propto h_j w_{ij}^2.$$

Ranking by Δ_{ij} is equivalent to ranking by

$$\sqrt{\Delta_{ij}} \propto |w_{ij}| \sqrt{h_j} \propto |w_{ij}| \|x_j\|_2,$$

which is exactly the Wanda score in Equation (50). Relation to the common framework: Wanda is a diagonal-Hessian simplification of the same second-order pruning logic. It sacrifices compensation for speed and simplicity.

15.8.3 LLM-Pruner: group-level Taylor pruning with fast LoRA recovery

LLM-Pruner [64] moves from individual weights to coupled structures such as heads, hidden dimensions, or block components. Introduce a gate $g_k \in \{0, 1\}$ for structure k , so the layer becomes $f(x; g) = f(x; \{g_k W_k\})$. Around $g_k = 1$, the first-order Taylor score is

$$I_k = \left| \frac{\partial \mathcal{L}}{\partial g_k} g_k \right|. \quad (51)$$

This is the group analogue of classical saliency: prune structures with small estimated effect on loss, then briefly recover with LoRA fine-tuning. Relation to the classical framework: LLM-Pruner replaces scalar saliency by group saliency and inserts cheap post-pruning adaptation. Its main claim is that task-agnostic structural pruning of LLMs is feasible with a small recovery budget rather than full retraining [64].

15.8.4 FLAP: retraining-free structured pruning with fluctuation-based importance

FLAP [65] targets structured pruning without retraining. The intuition is that removing a column is less harmful when its output contribution is stable or easily recoverable. A simplified fluctuation score for removable structure j can be written as

$$F_j = \mathbb{E}_{x \sim \mathcal{C}} \left[\|w_j x_j - \mathbb{E}[w_j x_j]\|_2^2 \right], \quad (52)$$

where $w_j x_j$ is the contribution of that structure to the output. FLAP standardizes such scores globally to decide how much to prune from each block, then adds bias-like compensation terms to recover the output statistics. Relation to the classical framework: FLAP is still minimizing output perturbation, but at a structured rather than elementwise level and with an explicit compensation step. Its main claim is that retraining-free structured pruning can outperform earlier structured baselines and give cleaner hardware speedups than unstructured sparsity [65].

15.8.5 SliceGPT: structured dimension reduction via PCA and computational invariance

SliceGPT [27] shifts from weight sparsity to *dimension* sparsity. The key observation is that transformer blocks admit output-preserving orthogonal transformations, analogous to those used in QuaRot. Once the internal coordinates are rotated, one can remove low-variance directions and thereby delete rows and columns from weight matrices.

Suppose $X \in \mathbb{R}^{n \times d}$ contains block activations. The best rank- k orthogonal projection in Frobenius norm solves

$$\arg \max_{Q^\top Q = I} \text{tr} \left(D_k Q^\top X^\top X Q D_k \right), \quad (53)$$

where $D_k = \text{diag}(1, \dots, 1, 0, \dots, 0)$ selects the first k coordinates. By the Rayleigh-Ritz theorem, the optimum is obtained by choosing the columns of Q as the eigenvectors of $X^\top X$ sorted by decreasing eigenvalue. Hence the retained directions are the principal components. After rotating

the network into this basis, deleting the low-energy coordinates corresponds to deleting rows and columns from the transformed weights. Relation to the classical framework: SliceGPT is structured pruning guided by activation covariance, but unlike ordinary channel pruning it uses orthogonal invariances to move the compressible directions into removable coordinates. The result is a smaller dense model, which is exactly why it yields more faithful speedups than unstructured sparsity [27].

A conceptually important emerging line explicitly includes measured latency and post-pruning tunability in the pruning score. Even when the specific algorithms are still evolving, the lesson is already clear: prune what saves real milliseconds, not only what looks small in a layer-local surrogate.

Table 12: Recent influential sparsity/pruning works in relation to the classical pruning framework.

Work	Year	Relation to classical framework	Main improvement and evidence
SparseGPT	2023	Row-wise OBS-style second-order reconstruction with activation-covariance Hessians	Demonstrates accurate one-shot pruning at LLM scale, including semi-structured settings
Wanda	2023–24	Diagonal-Hessian simplification of OBD/OBS for LLMs	Very simple score $ w_{ij} \ x_j\ _2$ with strong empirical performance and low cost
LLM-Pruner	2023	Group-level Taylor pruning plus short low-rank recovery tuning	Makes structural pruning practical without full retraining
FLAP	2024	Structured output-perturbation minimization with compensation and global allocation	Strong retraining-free structured pruning with hardware-friendly patterns
SliceGPT	2024	Structured pruning after PCA-like rotation into removable coordinates	Produces smaller <i>dense</i> models, hence cleaner real speedups than generic unstructured sparsity

Checkpoint. Unstructured pruning removes numbers from a matrix. Structured pruning removes entire columns, channels, heads, or sublayers. Only the second category usually yields direct speedups on mainstream runtimes unless the hardware explicitly supports a sparse pattern such as 2:4.

16 Distillation for deployable models, then low-rank structure and fast adaptation

16.1 First-pass picture: what distillation changes

Quantization and pruning start from a trained model and modify it. Distillation instead trains a *different* model—usually smaller, shallower, narrower, or architecturally simpler—to imitate a stronger teacher. This is why distillation often gives the cleanest real speedups in deployment: the student is dense and natively small, so standard optimized kernels still apply.

A practical mental model is:

- use **distillation** when you are allowed to change the deployed architecture;
- use **LoRA-like adaptation** when the deployed backbone must stay mostly fixed but task specialization is needed.

Table 13: What can be distilled from a teacher to a student?

Target	Typical loss	What it teaches the student
Final logits	KL / soft cross-entropy	The teacher’s class or token distribution
Hidden states	ℓ_2 or cosine loss	Intermediate representations and features
Attention relations	KL / MSE on attention maps or value relations	How the model routes information internally
Whole sequences	Sequence-level KL or pseudo-target training	How the teacher behaves under generation, not only under teacher forcing
Rationales / intermediate text	Token-level cross-entropy on explanations or chains of thought	Extra supervision about the teacher’s reasoning trace or decomposition of the task

16.2 Canonical knowledge distillation objective

Let z^T and z^S be teacher and student logits for the same input, and define temperature- τ probabilities

$$p_\tau^T = \text{softmax}(z^T/\tau), \quad p_\tau^S = \text{softmax}(z^S/\tau).$$

Standard distillation minimizes a mixture of hard-label supervision and teacher imitation:

$$\mathcal{L}_{\text{KD}} = (1 - \alpha) \text{CE}(y, p_1^S) + \alpha \tau^2 \text{KL}(p_\tau^T \| p_\tau^S). \quad (54)$$

Because cross-entropy with a soft target satisfies

$$\text{CE}(q, p) = H(q) + \text{KL}(q \| p),$$

and $H(q)$ is constant with respect to the student, minimizing soft-target cross-entropy is equivalent to minimizing forward KL.

16.3 Why the temperature-squared factor appears: full gradient derivation

For one example with soft target $q = p_\tau^T$, the distillation part is

$$L_{\text{soft}} = \text{KL}(q \| p_\tau^S) = \sum_i q_i \log \frac{q_i}{p_i}.$$

Since q is fixed,

$$\frac{\partial L_{\text{soft}}}{\partial z_i^S} = - \sum_j q_j \frac{1}{p_j} \frac{\partial p_j}{\partial z_i^S}.$$

For a temperature-softmax,

$$p_j = \frac{\exp(z_j^S / \tau)}{\sum_k \exp(z_k^S / \tau)}, \quad \frac{\partial p_j}{\partial z_i^S} = \frac{1}{\tau} p_j (\mathbf{1}\{i = j\} - p_i).$$

Substituting,

$$\frac{\partial L_{\text{soft}}}{\partial z_i^S} = \frac{1}{\tau} (p_i - q_i).$$

Thus the gradient magnitude has a visible $1/\tau$ factor. But as τ increases, the softened probabilities themselves become flatter and $p_i - q_i$ also shrinks by another factor of approximately $1/\tau$ for fixed logits. Hence the overall gradient scales like $1/\tau^2$. Multiplying by τ^2 in Equation (54) keeps the soft-target term numerically comparable across temperatures [30].

16.4 Historical derivation snapshot: FitNets and intermediate feature regression

FitNets [31] is historically important because it showed that distillation need not stop at the logits. Let h_T be a teacher hidden representation and h_S the corresponding student representation. If their dimensions differ, introduce a trainable regressor R so that $R(h_S)$ lives in the teacher’s space. A standard feature-distillation objective is

$$\mathcal{L}_{\text{FitNet}} = \mathcal{L}_{\text{task}} + \lambda \|R(h_S) - h_T\|_2^2. \quad (55)$$

The second term directly teaches the student an intermediate representation, not only the final answer. This idea later reappeared in TinyBERT-style hidden-state matching and in many multimodal and reasoning-distillation pipelines.

Plain-language interpretation. Logit KD says “copy the teacher’s final answer distribution.” FitNets says “also copy the teacher’s internal way of representing the problem.”

16.5 Historical derivation snapshot: relation distillation in transformers

MiniLM [35] made a different historical point: for transformers, one can distill not only hidden states but also *relations* between tokens. For one attention head, the teacher produces query, key, and value matrices Q_T, K_T, V_T , while the student produces Q_S, K_S, V_S . Define relation matrices

such as

$$R_T^{QK} = \frac{Q_T K_T^\top}{\sqrt{d_k}}, \quad R_T^{VV} = \frac{V_T V_T^\top}{\sqrt{d_v}},$$

and analogously for the student. A generic relation-distillation loss is then

$$\mathcal{L}_{\text{rel}} = \lambda_{QK} \left\| R_S^{QK} - R_T^{QK} \right\|_F^2 + \lambda_{VV} \left\| R_S^{VV} - R_T^{VV} \right\|_F^2. \quad (56)$$

This is historically important because it shows that the student can inherit *how tokens interact*, even if the student is smaller and cannot match all hidden states exactly.

Plain-language interpretation. FitNets teaches the student intermediate vectors. MiniLM-style relation distillation teaches the student the geometry of token-to-token interactions.

16.6 Sequence-level distillation and the train-inference mismatch

For autoregressive models,

$$P_\theta(y_{1:T} \mid x) = \prod_{t=1}^T p_\theta(y_t \mid x, y_{<t}).$$

Standard token-level distillation under teacher forcing minimizes token divergences on ground-truth or teacher-generated prefixes. However, at inference the student conditions on its *own* prefixes. This train-inference mismatch is one reason naive sequence distillation can produce brittle students.

A sequence-level forward-KL objective is

$$\text{KL}(P_T \parallel P_S) = \mathbb{E}_{y \sim P_T} \left[\sum_{t=1}^T \log \frac{p_T(y_t \mid x, y_{<t})}{p_S(y_t \mid x, y_{<t})} \right]. \quad (57)$$

Equation (57) shows the issue clearly: the expectation is over teacher trajectories. If the student later visits different prefixes, this objective gives no direct supervision there.

This motivates on-policy distillation: sample trajectories from the student, then compare teacher and student token distributions *along those student trajectories*. That change of state distribution is the conceptual core of modern LLM distillation.

16.7 Reverse KL and MiniLLM’s generative view

MiniLLM [39] argues that for generative LLM distillation, reverse KL is often better suited than forward KL:

$$\text{KL}(P_S \parallel P_T) = \mathbb{E}_{y \sim P_S} \left[\log \frac{P_S(y \mid x)}{P_T(y \mid x)} \right]. \quad (58)$$

Using the autoregressive factorization,

$$\log \frac{P_S(y \mid x)}{P_T(y \mid x)} = \sum_{t=1}^T \log \frac{p_S(y_t \mid x, y_{<t})}{p_T(y_t \mid x, y_{<t})}.$$

So

$$\text{KL}(P_S \| P_T) = \mathbb{E}_{y \sim P_S} \left[\sum_{t=1}^T \log \frac{p_S(y_t \mid x, y_{<t})}{p_T(y_t \mid x, y_{<t})} \right]. \quad (59)$$

This encourages the student to place mass where the teacher itself assigns high probability, rather than trying to cover the full teacher support. MiniLLM further decomposes the objective to reduce variance and mixes teacher-guided sampling to prevent degenerate trajectories [39]. Relation to the canonical framework: MiniLLM changes the divergence in Equation (54) to better match autoregressive generation.

16.8 Generalized knowledge distillation (GKD) and on-policy learning

GKD [40] abstracts on-policy distillation as

$$\mathcal{L}_{\text{GKD}} = \mathbb{E}_{(x,y) \sim \mathcal{B}} [D(P_T(\cdot \mid x, y_{<\cdot}) \| P_S(\cdot \mid x, y_{<\cdot}))], \quad (60)$$

where the batch $\mathcal{B}$ is drawn partly from data and partly from student-generated sequences. A simple mixture policy is controlled by a student-data fraction λ :

- with probability λ , sample $y \sim P_S(\cdot \mid x)$,
- otherwise use dataset or teacher sequences.

Forward KL, reverse KL, or other divergences can be used. Relation to the classical framework: GKD keeps the token-level divergence idea of Equation (54), but changes the *state distribution* on which the divergence is evaluated. This directly addresses the exposure-bias problem in Equation (57).

16.9 How to evaluate a distilled model for deployment

A distilled student should be judged on three axes simultaneously:

1. **Task quality:** not only perplexity, but also the downstream metrics that matter for deployment.
2. **Architecture efficiency:** parameter count, activation size, and KV-cache footprint of the student itself.
3. **Serving performance:** TTFT, TPOT, throughput, and memory on the same runtime stack used for the teacher baseline.

A common fallacy is to claim that distillation “worked” because the KL term fell or because perplexity stayed close to the teacher, even though the student architecture was not actually chosen to meet the target latency budget. For deployable students, the architecture and the loss must be designed together.

16.10 Low-rank compression: the Eckart-Young derivation

Now consider a weight matrix $W \in \mathbb{R}^{m \times n}$. The classical low-rank compression problem is

$$\arg \min_{\text{rank}(\tilde{W}) \leq r} \|W - \tilde{W}\|_F^2. \quad (61)$$

Let the singular value decomposition be

$$W = U \Sigma V^\top, \quad \Sigma = \text{diag}(\sigma_1, \dots, \sigma_p), \quad \sigma_1 \geq \sigma_2 \geq \dots.$$

Because the Frobenius norm is orthogonally invariant,

$$\|W - \tilde{W}\|_F = \|\Sigma - U^\top \tilde{W} V\|_F.$$

Thus the optimal rank- r approximation is obtained by keeping the top r singular values and zeroing the rest:

$$\tilde{W} = U_r \Sigma_r V_r^\top. \quad (62)$$

The approximation error is

$$\|W - \tilde{W}\|_F^2 = \sum_{i>r} \sigma_i^2.$$

Equation (62) is the Eckart-Young theorem and the root of low-rank compression. Computationally, the linear layer becomes

$$Wx \approx U_r(\Sigma_r(V_r^\top x)),$$

turning one large matrix multiply into two smaller ones. This reduces parameters and, when r is small enough, can reduce compute as well.

16.11 LoRA as constrained low-rank adaptation

LoRA [38] starts from the observation that the fine-tuning update ΔW often has low intrinsic rank. The ideal constrained problem is

$$\min_{\Delta W} \mathcal{L}(W_0 + \Delta W) \quad \text{subject to} \quad \text{rank}(\Delta W) \leq r. \quad (63)$$

Instead of solving Equation (63) directly, LoRA parameterizes

$$\Delta W = BA, \quad B \in \mathbb{R}^{m \times r}, \quad A \in \mathbb{R}^{r \times n}, \quad (64)$$

often with a scaling factor α/r :

$$W = W_0 + \frac{\alpha}{r} BA. \quad (65)$$

The linear layer output is then

$$y = W_0 x + \frac{\alpha}{r} B(Ax).$$

At inference, the adapter can be merged into the base weight:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r}BA.$$

This is crucial for deployment: after merging, LoRA adds no extra inference latency relative to the base architecture.

16.12 Fast adaptation on quantized backbones: QLoRA and LoftQ

QLoRA [42] combines low-rank adaptation with 4-bit storage. Its NormalFloat- k data type is derived from quantiles of a standard normal distribution. For 2^k levels, one idealized codebook construction is

$$q_i = \frac{1}{2} \left[Q_{\mathcal{N}(0,1)}\left(\frac{i}{2^k+1}\right) + Q_{\mathcal{N}(0,1)}\left(\frac{i+1}{2^k+1}\right) \right], \quad (66)$$

followed by normalization to $[-1, 1]$. This makes the codebook particularly well matched to normally distributed pretrained weights.

QLoRA then stores the base weight in NF4, dequantizes to BF16 for computation, and trains only the low-rank adapter:

$$Y_{\text{BF16}} = X_{\text{BF16}} \text{doubleDequant}(c_1, c_2, W_{\text{NF4}}) + X_{\text{BF16}}L_1L_2. \quad (67)$$

Here c_1 and c_2 are first- and second-level quantization constants, and “double quantization” reduces the storage cost of those constants themselves. Relation to the common framework: QLoRA combines low-rank adaptation with weight quantization, but only the adapter receives gradients.

LoftQ [43] goes one step further by jointly choosing the quantized backbone and the initial low-rank adapter. Its core problem is

$$\min_{\hat{W} \in \mathcal{Q}, A, B} \|W - \hat{W} - AB\|_F^2. \quad (68)$$

This admits a natural alternating minimization:

1. Fix A, B , quantize the residual $W - AB$ to get $\hat{W}$.
2. Fix $\hat{W}$, compute the best rank- r approximation of $W - \hat{W}$ via truncated SVD to get AB .
3. Iterate.

Relation to the classical framework: LoftQ fuses Equation (61) with a discrete quantization constraint. Its main contribution is to reduce the initialization mismatch between a quantized base model and the low-rank correction [43].

16.13 Adaptive rank allocation and weight decomposition: AdaLoRA and DoRA

AdaLoRA [44] notes that not all layers deserve the same adaptation rank. It parameterizes the update in SVD-like form and reallocates budget to more important matrices over training. Conceptually, it replaces the fixed-rank constraint in Equation (63) by a *budgeted* allocation problem across layers.

DoRA [45] asks a different question: perhaps LoRA underuses capacity because it asks the same low-rank update to modify both weight magnitude and direction. Write each column of a weight matrix as

$$w_j = m_j v_j, \quad m_j = \|w_j\|_2, \quad \|v_j\|_2 = 1.$$

Stacking columns gives a decomposition

$$W = \text{diag}(m)V,$$

with V column-normalized. DoRA keeps a trainable magnitude vector m while applying LoRA to the directional part V . This more closely matches full fine-tuning behavior while remaining mergeable before inference. Relation to the common framework: DoRA is still a low-rank adaptation method, but it changes the parameterization to improve optimization and expressivity [45].

16.14 Historically important works from root to current practice

Model compression by teacher-student transfer predates modern deep learning [29]. Hinton et al. popularized modern distillation [30], while FitNets, sequence-level KD, DistilBERT, TinyBERT, and MiniLM refined intermediate-layer matching and transformer distillation [31, 32, 33, 34, 35]. In parallel, low-rank structure was studied in neural network parameter matrices and convolutional speedups [36, 37]. LoRA then became the dominant practical PEFT method because it is train-efficient and mergeable [38].

Table 14: Historical roots of distillation and low-rank adaptation.

Work	Year	Key idea	Why it still matters
Model compression	2006	Teacher-student transfer as a compression primitive	Established the teacher/student framing before modern KD
Hinton et al.	2015	Temperature-smoothed KD	Canonical KL-based distillation objective
FitNets	2015	Intermediate feature matching	Shows that hidden-state supervision can help small students
Sequence KD	2016	Distill whole sequences rather than isolated tokens	Important bridge to autoregressive generation
MiniLM	2020	Attention/relation distillation for Transformers	Strong bridge from BERT compression to LLM-style distillation
LoRA	2022	Low-rank parameter-efficient updates	Default starting point for fast adaptation of deployable models

16.15 Recent influential works

16.15.1 MiniLLM: reverse-KL distillation for generative students

MiniLLM [39] replaces the forward-KL view of Equation (54) by the reverse-KL view of Equation (58), then develops variance-reduced optimization. The paper’s main argument is that forward KL pushes

a low-capacity student to cover low-probability teacher regions, while reverse KL is mode-seeking and therefore better matches the realities of generative model compression. The reported gains include better calibration, lower exposure bias, and stronger long-form generation.

16.15.2 GKD: distillation on student-generated trajectories

GKD [40] makes the training distribution a first-class design choice. Its generalized framework mixes dataset sequences and student-generated sequences and allows multiple divergences. Relative to the canonical KD framework, it changes *where* the divergence is enforced, not just *which* divergence is used. This is why it improves over standard supervised distillation on auto-regressive tasks.

16.15.3 Distilling Step-by-Step: rationales as intermediate supervision

Distilling Step-by-Step [41] augments distillation targets with teacher-generated rationales. A simple abstraction is a multi-task loss

$$\mathcal{L} = \lambda_y \text{CE}(y, \hat{y}) + \lambda_r \text{CE}(r, \hat{r}),$$

where r is a rationale sequence. Relation to the common framework: this enriches the target in Equation (54) with intermediate reasoning structure. The practical appeal is deployment: the rationale is used during training, but the deployed student does not need the teacher.

16.15.4 QLoRA: quantized storage plus low-rank training

QLoRA [42] is influential because it made high-quality adaptation of large models feasible on modest hardware. The method is not primarily an inference-speed method, but it is central for *fast adaptation of deployable models*. Its key claim is that a 4-bit stored base model with BF16 compute and LoRA updates can approach the quality of 16-bit adapter tuning while drastically reducing memory.

16.15.5 LoftQ, AdaLoRA, and DoRA: three directions beyond vanilla LoRA

LoftQ [43] optimizes the quantized base and low-rank initialization jointly via Equation (68). AdaLoRA [44] reallocates rank budget where it is most useful. DoRA [45] decomposes weight updates into magnitude and direction to better match full fine-tuning. All three are direct descendants of the constrained low-rank viewpoint in Equation (63), but each improves a different weakness: quantization mismatch, budget uniformity, or parameterization quality.

Checkpoint. Distillation changes the student model itself; low-rank adaptation changes only how the update is represented. For deployment, distill when you are allowed to change the architecture. Use LoRA-like methods when you must stay close to the base model but still want cheap task adaptation.

Table 15: Recent distillation and low-rank adaptation works.

Work	Year	Relation to the common framework	Main contribution
MiniLLM	2023–24	Changes forward KL to reverse KL for generative distillation	Better mode-seeking student behavior and stronger long-form generation
GKD	2024	Enforces distillation on student-generated trajectories	Reduces train-inference mismatch in autoregressive students
Distilling Step-by-Step	2023	Adds rationale supervision to teacher targets	Stronger small students with less labeled data
QLoRA	2023	Quantized base model plus low-rank adapters	Cheap adaptation of large deployable models
LoftQ	2023	Joint quantization and low-rank initialization	Better quantized-adapter starting point, especially at low bits
AdaLoRA	2023	Adaptive rank-budget allocation across layers	More efficient use of adaptation parameters
DoRA	2024	Weight decomposition into magnitude and direction	Better optimization and stronger PEFT while remaining mergeable

17 Benchmarking latency and throughput correctly

17.1 The right metrics

For modern inference, the minimum reporting set is:

- **End-to-end request latency:** wall-clock time from request arrival to final output.
- **TTFT (time to first token):** especially relevant for interactive LLM use.
- **TPOT / ITL (time per output token / inter-token latency):** decode efficiency after the first token.
- **Throughput:** requests/s or tokens/s under a stated concurrency.
- **Goodput:** throughput subject to latency SLOs.
- **Memory footprint:** model weights plus KV cache and runtime overhead.
- **Energy:** joules/request or tokens/joule, increasingly important in production.

For a batch of completed requests $\mathcal{R}$ over wall time T_{wall} ,

$$\text{throughput}_{\text{req/s}} = \frac{|\mathcal{R}|}{T_{\text{wall}}}, \quad \text{throughput}_{\text{tok/s}} = \frac{\sum_{r \in \mathcal{R}} L_r^{\text{out}}}{T_{\text{wall}}}.$$

A useful online metric is goodput:

$$\text{goodput} = \frac{\#\{r \in \mathcal{R} : \text{TTFT}_r \leq \tau_1, \text{TPOT}_r \leq \tau_2\}}{T_{\text{wall}}}.$$

17.2 Latency decomposition and why averages are not enough

Equation (6) already gives the main decomposition:

$$T_{\text{req}} = T_{\text{queue}} + T_{\text{prefill}} + \sum_{t=1}^{L_{\text{out}}} T_{\text{decode},t} + T_{\text{post}}.$$

For many LLMs:

- **Prefill** is dominated by large batched matrix multiplications and attention over the prompt.
- **Decode** is dominated by repeated smaller matmuls plus growing KV-cache reads/writes.

A method that speeds up prefill may barely change TPOT; a method that shrinks the KV cache may primarily improve decode. Therefore one must not report only one scalar latency number without sequence lengths and workload composition.

17.3 Roofline and queueing: the two views every benchmark should include

The kernel-level roofline view was given in Equation (5). At the service level, Little’s law adds the queueing perspective:

$$L = \lambda W, \tag{69}$$

where L is average concurrency in the system, λ is throughput, and W is average latency [1, 2]. Equation (69) explains a common confusion: a system can achieve high throughput only by operating at a concurrency point where latency is much worse than single-stream latency. Therefore “best latency” and “max throughput” are different operating points and should not be mixed in a single chart without explanation.

17.4 IO-aware runtime optimization: FlashAttention as a worked example

Standard attention is

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V, \quad Q, K, V \in \mathbb{R}^{n \times d}.$$

The naive implementation materializes the score matrix $S = QK^\top/\sqrt{d} \in \mathbb{R}^{n \times n}$ and often also the probability matrix $P = \text{softmax}(S)$. That creates $O(n^2)$ intermediate memory traffic even though the final output is only $n \times d$. If attention is bandwidth-bound, the correct optimization target is therefore IO, not only FLOPs.

The key algebraic trick is the online softmax recurrence. Suppose one query row attends to keys in

tiles. After processing tiles $1, \dots, t$, maintain three quantities: a running maximum $m^{(t)}$, a running normalizer $\ell^{(t)}$, and a running output $o^{(t)}$. For the next tile with scores $s^{(t+1)}$, update

$$m^{(t+1)} = \max(m^{(t)}, \max s^{(t+1)}), \quad (70)$$

$$\ell^{(t+1)} = e^{m^{(t)} - m^{(t+1)}} \ell^{(t)} + \sum_j e^{s_j^{(t+1)} - m^{(t+1)}}, \quad (71)$$

$$o^{(t+1)} = \frac{e^{m^{(t)} - m^{(t+1)}} \ell^{(t)} o^{(t)} + \sum_j e^{s_j^{(t+1)} - m^{(t+1)}} V_j^{(t+1)}}{\ell^{(t+1)}}. \quad (72)$$

Equations (70)–(72) compute the exact softmax attention output without storing the full $n \times n$ matrix in high-bandwidth memory. FlashAttention turns this algebra into a tiled kernel that keeps intermediates in SRAM [66]. FlashAttention-2 keeps the same exact computation but improves work partitioning and parallelism so attention gets much closer to GEMM efficiency [67].

Relation to benchmarking: if a serving stack silently switches between naive attention and FlashAttention-style kernels, the measured effect of quantization or pruning may change substantially. Runtime kernels are therefore part of the benchmark definition, not an implementation detail.

17.5 Speculative decoding: fewer target-model passes without changing the target distribution

Autoregressive decoding is slow because the target model normally performs one serial forward pass per generated token. Speculative decoding [46, 47] introduces a cheaper draft model q that proposes several tokens at once, while the expensive target model p verifies them in parallel.

Suppose the draft proposes a block $\tilde{y}_{1:\gamma}$. The target model computes the conditional probabilities $p_i(\cdot)$ for each position in that block in one pass. For each proposed token $\tilde{y}_i$, accept it with probability

$$a_i = \min\left(1, \frac{p_i(\tilde{y}_i)}{q_i(\tilde{y}_i)}\right). \quad (73)$$

If the first rejection occurs at position j , one samples a correction token from the normalized positive part of the difference between the target and draft distributions:

$$r_j(v) \propto [p_j(v) - q_j(v)]_+. \quad (74)$$

If all γ proposals are accepted, one may additionally sample one fresh token from the target model. Equations (73)–(74) are a rejection-sampling construction; they guarantee that the final sampled sequence still follows the target distribution p exactly.

If the expected number of accepted draft tokens per target pass is $\bar{k}$, then the number of expensive target passes per generated token drops from about 1 to about $1/\bar{k}$. This is why speculative decoding can materially reduce decode latency even though the target model itself is unchanged.

Plain-language interpretation. A cheap helper model guesses a few future tokens. The large model checks those guesses in parallel. As long as the acceptance-correction rule is used, the final output distribution is still exactly the large model’s.

17.6 PagedAttention and continuous batching: why KV-cache management changes throughput

For long-context LLM serving, memory fragmentation can be as important as arithmetic. Suppose a contiguous allocator reserves memory for up to $T_{\max}$ cached tokens per active sequence. If a given sequence currently uses only T_s cached tokens, the wasted allocation is

$$(T_{\max} - T_s) \cdot b_{KV}$$

bytes, where b_{KV} is the bytes per cached token. When many sequences have different current lengths, this waste can be large.

PagedAttention [51] replaces contiguous reservation by fixed-size pages of P tokens. A sequence of current length T_s then needs

$$n_s = \left\lceil \frac{T_s}{P} \right\rceil$$

pages, so the internal fragmentation is

$$n_s P - T_s < P. \tag{75}$$

Equation (75) is the key systems result: wasted cache per sequence is bounded by one page, rather than by the gap to a worst-case reserved context length. This lower fragmentation allows more active sequences to fit in memory, which in turn allows more aggressive *continuous batching*. Under Little’s law, higher steady-state concurrency can raise throughput substantially even if the underlying model weights do not change.

Plain-language interpretation. PagedAttention does for KV-cache memory what virtual memory does for operating systems: it avoids forcing each request to own one big contiguous chunk of memory.

17.7 A correct measurement protocol

A trustworthy benchmark usually follows this order:

1. Fix the hardware, driver, compiler/runtime version, and serving framework.
2. Fix the workload: prompt length distribution, output length distribution, batch size, concurrency, decoding parameters, and tokenizer.
3. Warm up the model, kernels, and graph captures.
4. Time the *steady-state* region only, unless startup is explicitly part of the target use case.

5. Synchronize correctly when measuring accelerators.
6. Report p50/p95/p99 where relevant, not just means.
7. Separate prefill and decode for generative models.
8. Verify numerics and task quality on the same workload.
9. Check that the claimed kernel path is actually used.
10. Repeat enough times to estimate variability.

17.8 Common measurement fallacies

Table 16: Measurement fallacies and the correct fix.

Fallacy	Corrective action
Comparing different prompt lengths, output lengths, or decoding settings	Fix the exact workload and report it explicitly
Reporting sparse FLOP reduction as if it were latency reduction	Benchmark end-to-end on the actual sparse runtime and hardware
Timing GPU kernels without synchronization	Use proper synchronization or device events
Including startup or excluding warmup selectively	Separate cold-start, warmup, and steady-state numbers
Ignoring kernel fallback from low-bit or sparse formats to dense/high-precision compute	Inspect the runtime path and confirm the intended kernels execute
Reporting average latency only	Include p50/p95/p99, especially for online serving
Comparing different software stacks or scheduler policies	Keep the serving stack constant when attributing gains to the model method
Conflating offline throughput with interactive latency	Report both separately; they are different operating points
Ignoring host overhead, packing, dequantization, and KV-cache traffic	Use end-to-end timings, not isolated GEMM microbenchmarks alone
Claiming quality preservation from perplexity only	Also report task-level accuracy or downstream behavior on representative tasks

17.9 Recent influential works on measurement and serving

17.9.1 FlashAttention / FlashAttention-2: IO-aware kernels can beat many model-side tricks

FlashAttention [66] showed that exact attention can be accelerated by reducing HBM traffic rather than approximating the model. FlashAttention-2 [67] preserved exact attention but improved work partitioning and parallelism so the kernel gets much closer to GEMM efficiency. Relation to the unified framework: these methods change $T(\tilde{\theta}, \pi, \mathcal{H})$ by changing the kernel implementation, not the model. Their community impact is large because many later serving stacks assume IO-aware attention kernels by default.

17.9.2 vLLM / PagedAttention: systems optimization can dominate model-side gains

The vLLM/PagedAttention work [51] is influential because it shows that serving-systems engineering can yield order-one gains without changing the model. The key ideas are page-like KV-cache management, low fragmentation, and continuous batching of heterogeneous requests. Relation to the unified framework: this work changes the runtime policy π in Equation (1), not $\tilde{\theta}$. The paper reports roughly 2–4 $\times$ throughput gains at similar latency for popular LLM serving scenarios.

17.9.3 MLPerf Inference: standardized measurement matters

MLPerf Inference provides standardized load-generation methodology, scenario definitions, latency constraints, and comparable reporting [49, 50]. Relation to the common framework: MLPerf does not invent a new model optimization, but it makes the functions T and Q in Equation (1) measurable in a controlled and comparable way. This matters because many apparent efficiency gains vanish when measured under a common protocol.

17.9.4 Speculative decoding: exact acceleration of autoregressive sampling

Speculative decoding [46, 47] is influential because it attacks the serial decode bottleneck directly. Relative to the common framework, it changes the runtime policy π while preserving the target model’s output distribution. Its practical importance is that it can speed up decoding without changing the deployed target model’s parameters or the exact sampling law.

17.9.5 Medusa: in-model speculation without a separate draft model

Medusa [48] replaces the external draft model by extra decoding heads attached to the backbone. These heads propose multiple future tokens, which the backbone then verifies with a tree-style attention pattern. Relative to the common framework, Medusa is a lightweight architectural extension aimed directly at decode-time throughput. The paper reports multi-token acceptance and substantial speedups, especially when maintaining a second draft model would be operationally inconvenient.

Table 17: Recent influential serving and benchmarking works.

Work	Year	Relation to efficiency	Main lesson
FlashAttention / 2	2022–23	IO-aware attention kernels	Runtime kernels can remove dominant memory traffic without changing the model function
vLLM / PagedAttention	2023	KV-cache paging plus continuous batching	Service engineering can deliver order-one throughput gains without modifying weights
Speculative decoding	2023	Exact runtime policy for fewer target-model passes	Decode can be accelerated while preserving the target distribution
Medusa	2024	Lightweight multi-head speculation	In-model speculation can reduce decode steps without maintaining a separate draft model
MLPerf Inference	2020–25	Standardized benchmark scenarios and reporting	Comparable efficiency claims require controlled scenarios, load generation, and latency accounting

Checkpoint. A model can stay mathematically identical and still become much faster if the runtime changes how it uses memory. This is why benchmarking must specify the attention kernel, the KV-cache manager, and the scheduler—not only the model weights.

18 A practical synthesis: what usually works in deployment

A pragmatic deployment sequence often looks like this:

Step 1: benchmark the dense baseline and identify the real bottleneck. Separate prefill and decode, record memory, p95 latency, throughput, and energy, and decide whether the system is limited by compute, bandwidth, KV-cache capacity, or scheduler/queueing effects.

Step 2: fix obvious runtime inefficiencies before compressing the model. If the workload is long-context or throughput-oriented, mature runtime optimizations such as FlashAttention-class kernels and paged KV-cache serving can yield larger gains than aggressive model surgery.

Step 3: try quantization first. It is usually the fastest route to lower memory and better bandwidth efficiency. Start with the hardware-native format: often INT8, W8A8, or weight-only INT4/INT8 depending on the target accelerator.

Step 4: if quantization error is too large, decide between better PTQ and QAT. Better PTQ means improved scaling, reconstruction, or rotation (AWQ, SmoothQuant, OmniQuant, QuaRot, SpinQuant). QAT is justified when the target bitwidth is aggressive or the task is too sensitive for PTQ.

Step 5: use sparsity only with a hardware story. If the target runtime lacks sparse kernels, prefer structured pruning or dimension reduction (for example SliceGPT-like methods) over generic unstructured sparsity.

Step 6: if architecture change is allowed, distill. Distillation usually gives the best accuracy-latency frontier because the student is smaller by design, rather than merely compressed after the fact.

Step 7: use low-rank adaptation for fast task specialization. LoRA, DoRA, AdaLoRA, QLoRA, and LoftQ are especially useful when the deployment architecture must stay close to the base model, but task-specific adaptation is still required.

Table 18: Rule-of-thumb method selection.

Deployment need	Most natural first move
Need an immediate memory reduction with minimal engineering	PTQ, preferably hardware-native INT8 or weight-only low-bit
Long-context or throughput-limited LLM serving	Fix the runtime first: IO-aware attention kernels and efficient KV-cache management
Need very aggressive low-bit inference and can afford some calibration/search	Advanced PTQ such as AWQ, SmoothQuant, OmniQuant, QuaRot, SpinQuant
Need strong latency gains on standard dense runtimes	Structured pruning or dimension reduction, not generic unstructured sparsity
Need the best quality-speed trade-off and can retrain a student	Distillation
Need cheap task adaptation with mergeable inference path	LoRA / DoRA / AdaLoRA
Need cheap adaptation on limited-memory hardware	QLoRA or LoftQ-style quantized adaptation
Need trustworthy efficiency claims	Standardized, synchronized, end-to-end benchmarking with workload disclosure

19 Conclusion

The modern efficiency toolbox is much easier to understand once everything is viewed through two lenses: a *compression lens*, in which one solves a constrained approximation or retraining problem, and a *systems lens*, in which one must improve the actual bottleneck on the target serving stack. Quantization changes numeric representation; pruning changes topology or sparsity pattern; distillation changes the training target so a smaller model can exist at all; low-rank methods constrain how change is represented; benchmarking decides whether any of those changes matter in practice.

Historically, the field moved from classical local-saliency and low-precision ideas to practical deep compression, then to large-model PTQ and PEFT, and most recently to deployment-aware methods

that explicitly co-design compression with hardware realities. The main operational lesson is straightforward. If retraining is limited, begin with PTQ and careful benchmarking. If true speedup is mandatory, use structured methods or distill a smaller student. If fast adaptation is needed, use low-rank updates that can be merged before inference. And in every case, distrust claims that stop at parameter count or FLOPs: the only efficiency improvement that finally matters is the one that survives end-to-end measurement.

A Historical lineage at a glance

Table 19: From roots to current practice.

Theme	Foundational roots	Middle generation	Recent LLM-era branch
Quantization	Binarized / quantized neural nets; integer-only inference [3, 4, 5]	PACT, LSQ, HAWQ, AdaRound, BRECQ [6, 54, 55, 56, 8, 57]	ZeroQuant, GPTQ, SmoothQuant, AWQ, OmniQuant, LLM-QAT, BitDistiller, QuaRot, SpinQuant [58, 9, 11, 12, 60, 61, 62, 13, 14]
Pruning / sparsity	OBD, OBS [15, 16]	Deep Compression, filter/channel pruning, state-of-sparsity work [17, 18, 19, 20, 23]	SparseGPT, Wanda, LLM-Pruner, FLAP, SliceGPT, latency-aware pruning [25, 26, 64, 65, 27, 28]
Distillation	Model compression via teachers [29]	Hinton KD, Fit-Nets, sequence KD, DistilBERT/TinyBERT/MiniLM [30, 31, 32, 33, 34, 35]	MiniLLM, GKD, Distilling Step-by-Step [39, 40, 41]
Low-rank structure	Early low-rank redundancy in neural parameters [36, 37]	PEFT via LoRA [38]	QLoRA, LoftQ, AdaLoRA, DoRA [42, 43, 44, 45]
Benchmarking / runtime	Queueing and roofline principles [1, 2]	Standardized benchmark methodology [49]	FlashAttention/2, speculative decoding, Medusa, vLLM, MLPerf 5.x [66, 67, 46, 48, 51, 50]

B Glossary and FAQ

B.1 Glossary

Table 20: A compact glossary for readers returning to the note after a break.

Term	Meaning in this note
Activation	The output of a layer for a given input. Activations are data-dependent, unlike weights, which are fixed after training.
Calibration set	A small representative dataset used by PTQ methods to estimate activation ranges or reconstruction error without full retraining.
Channel	One structured slice of a tensor, often corresponding to one output feature map or one output dimension block. Structured pruning often removes channels.
Clipping	Limiting a value to stay inside an allowed range. In quantization, clipping protects the integer range but can introduce error for outliers.
Concurrency	How many requests are active at the same time in a serving system. Throughput and latency both depend on it.
Decode	The autoregressive generation phase in which the model produces one new token at a time.
Distillation	Training a smaller student model to imitate a larger teacher model.
Fake quantization	A training-time operator that simulates rounding and clipping while still allowing gradient-based learning of full-precision parameters.
Group-wise quantization	Quantization in which one scale is shared by a small group of weights rather than by the whole tensor or by one channel.
Hessian	The matrix of second derivatives of the loss with respect to parameters. It measures local curvature.
Kernel	A low-level implementation of an operation on hardware, such as a matrix multiply kernel. Numerical schemes help only when efficient kernels exist.
KV cache	The stored keys and values from previous tokens used by transformer decoding to avoid recomputation.
Latency	The time taken for a single request to finish.
Low rank	A matrix has low rank if only a few independent directions explain most of its variation.
Output reconstruction	A surrogate objective that measures how well a modified layer reproduces the original layer output on calibration data.

Term	Meaning in this note
Per-channel quantization	A quantization scheme that uses a separate scale for each channel, usually improving accuracy relative to one global scale.
Perplexity	A standard language-model quality metric based on the average negative log-likelihood; lower is better.
Prefill	The phase that processes the prompt tokens and builds the initial KV cache before generation begins.
Saliency	A score estimating how important a weight or group is for the loss. OBD and OBS define saliency using second-order approximations.
Semi-structured sparsity	A regular sparsity pattern such as 2 : 4, where each local group must contain a fixed number of nonzeros.
Straight-through estimator (STE)	A surrogate gradient rule that treats a non-differentiable operation such as rounding as if its derivative were simple, often approximately one in a trusted region.
Structured sparsity	Sparsity that removes entire rows, columns, heads, channels, or blocks rather than arbitrary isolated weights.
Throughput	The rate at which a serving system finishes work, often measured in requests/s or tokens/s.
TTFT	Time to first token. An LLM-specific latency metric.
TPOT	Time per output token after the first token. Another LLM-specific latency metric.

B.2 FAQ

Why does this note keep separating “model-side” and “runtime-side” ideas?

Because they solve different problems. Quantization, pruning, distillation, and low-rank methods change the model. FlashAttention, speculative decoding, and PagedAttention change how the model is executed. In a real deployment, both layers interact.

Why is the same method sometimes described as compression and sometimes as acceleration?

Compression refers to reducing storage or representation cost. Acceleration refers to lowering wall-clock time. They overlap, but not perfectly. A compressed model can still be slow if it is executed badly, and a runtime optimization can accelerate a model without changing its parameter count at all.

When should distillation be preferred over quantization?

Choose quantization when you already have a trained model and want a cheap post-processing path. Choose distillation when you are allowed to train a smaller student and care strongly

about the quality-latency frontier of the final deployable model.

Does LoRA always add inference overhead?

Not always. If the learned low-rank update is merged into the base weights after training, inference can look almost like ordinary dense inference. If adapters remain separate at runtime, there can be some overhead.

References

- [1] J. D. C. Little. A proof for the queueing formula: $L = \lambda W$. *Operations Research*, 1961.
- [2] S. Williams, A. Waterman, and D. Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 2009.
- [3] M. Courbariaux, Y. Bengio, and J.-P. David. Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1. *NeurIPS*, 2016.
- [4] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio. Quantized neural networks: Training neural networks with low precision weights and activations. *Journal of Machine Learning Research*, 2017.
- [5] B. Jacob, S. Kligys, B. Chen, et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. *CVPR*, 2018.
- [6] J. Choi, Z. Wang, S. Venkataramani, P. Chuang, V. Srinivasan, and K. Gopalakrishnan. PACT: Parameterized clipping activation for quantized neural networks. *arXiv:1805.06085*, 2018.
- [7] R. Banner, Y. Nahshan, E. Hoffer, and D. Soudry. ACIQ: Analytical clipping for integer quantization of neural networks. *arXiv:1810.05723*, 2019.
- [8] M. Nagel, M. van Baalen, T. Blankevoort, and M. Welling. Up or down? Adaptive rounding for post-training quantization. *ICML*, 2020.
- [9] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv:2210.17323*, 2022.
- [10] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv:2208.07339*, 2022.
- [11] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. *ICML*, 2023.
- [12] J. Lin, J. Tang, H. Tang, et al. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *MLSys*, 2024; arXiv preprint 2023.
- [13] S. Ashkboos, A. Mohtashami, M. L. Croci, et al. QuaRot: Outlier-free 4-bit inference in rotated LLMs. *NeurIPS*, 2024.

-
- [14] Z. Liu, C. Zhao, I. Fedorov, et al. SpinQuant: LLM quantization with learned rotations. *arXiv:2405.16406*, 2024.
 - [15] Y. LeCun, J. S. Denker, and S. A. Solla. Optimal brain damage. *NeurIPS*, 1990.
 - [16] B. Hassibi and D. G. Stork. Second order derivatives for network pruning: Optimal brain surgeon. *NeurIPS*, 1993.
 - [17] S. Han, J. Pool, J. Tran, and W. Dally. Learning both weights and connections for efficient neural networks. *NeurIPS*, 2015.
 - [18] S. Han, H. Mao, and W. J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. *ICLR*, 2016.
 - [19] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf. Pruning filters for efficient ConvNets. *ICLR*, 2017.
 - [20] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang. Learning efficient convolutional networks through network slimming. *ICCV*, 2017.
 - [21] J. Frankle and M. Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *ICLR*, 2019.
 - [22] V. Sanh, T. Wolf, and A. Rush. Movement pruning: Adaptive sparsity by fine-tuning. *NeurIPS*, 2020.
 - [23] T. Gale, E. Elsen, and S. Hooker. The state of sparsity in deep neural networks. *arXiv:1902.09574*, 2019.
 - [24] A. Mishra, E. Elsen, C. Loh, et al. Accelerating sparse deep neural networks. *arXiv:2104.08378*, 2021.
 - [25] E. Frantar and D. Alistarh. SparseGPT: Massive language models can be accurately pruned in one-shot. *ICML*, 2023.
 - [26] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter. A simple and effective pruning approach for large language models. *ICLR*, 2024.
 - [27] S. Ashkboos, M. L. Croci, M. G. do Nascimento, T. Hoefer, and J. Hensman. SliceGPT: Compress large language models by deleting rows and columns. *ICLR*, 2024.
 - [28] S. Park, J. Kim, J. Lee, et al. Accurate sublayer pruning for large language models by exploiting latency and tunability information. *arXiv:2506.03510*, 2025.
 - [29] C. Bucilua, R. Caruana, and A. Niculescu-Mizil. Model compression. *KDD*, 2006.
 - [30] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015.
 - [31] A. Romero, N. Ballas, S. E. Kahou, et al. FitNets: Hints for thin deep nets. *ICLR*, 2015.

-
- [32] Y. Kim and A. Rush. Sequence-level knowledge distillation. *EMNLP*, 2016.
 - [33] V. Sanh, L. Debut, J. Chaumond, and T. Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *NeurIPS EMC2 Workshop*, 2019.
 - [34] X. Jiao, Y. Yin, L. Shang, et al. TinyBERT: Distilling BERT for natural language understanding. *Findings of EMNLP*, 2020.
 - [35] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *NeurIPS*, 2020.
 - [36] M. Denil, B. Shakibi, L. Dinh, N. de Freitas. Predicting parameters in deep learning. *NeurIPS*, 2013.
 - [37] E. Denton, W. Zaremba, J. Bruna, Y. LeCun, and R. Fergus. Exploiting linear structure within convolutional networks for efficient evaluation. *NeurIPS*, 2014.
 - [38] E. J. Hu, Y. Shen, P. Wallis, et al. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.
 - [39] Y. Gu, L. Dong, F. Wei, and M. Huang. MiniLLM: Knowledge distillation of large language models. *ICLR*, 2024.
 - [40] R. Agarwal, N. Vieillard, Y. Zhou, et al. On-policy distillation of language models: Learning from self-generated mistakes. *ICLR*, 2024.
 - [41] C.-Y. Hsieh, Y. Li, M. P. Bansal, et al. Distilling step-by-step! Outperforming larger language models with less training data and smaller model sizes. *arXiv:2305.02301*, 2023.
 - [42] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *NeurIPS*, 2023.
 - [43] Y. Li, Y. Yu, C. Liang, et al. LoftQ: LoRA-fine-tuning-aware quantization for large language models. *NeurIPS*, 2023.
 - [44] Q. Zhang, M. Chen, A. Bukharin, et al. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. *ICLR*, 2023.
 - [45] S.-Y. Liu, C.-Y. Wang, H. Yin, et al. DoRA: Weight-decomposed low-rank adaptation. *ICML*, 2024.
 - [46] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. *ICML*, 2023.
 - [47] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling. *arXiv:2302.01318*, 2023.
 - [48] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. *ICML*, 2024.
 - [49] P. Mattson, G. Cheng, C. Diamos, et al. MLPerf inference benchmark. *arXiv:1911.02549*, 2020.

-
- [50] MLCommons. MLPerf Inference v5.1 results and benchmark pages. 2025. Available from <https://mlcommons.org/benchmarks/inference-datacenter/> and related MLPerf Inference v5.1 release pages.
 - [51] W. Kwon, Z. Li, S. Zhuang, et al. Efficient memory management for large language model serving with PagedAttention. *arXiv:2309.06180*, 2023.
 - [52] M. Lazuka, A. Anghel, and T. Parnell. LLM-Pilot: Characterize and optimize performance of your LLM inference services. *SC*, 2024.
 - [53] J.-W. Chung, J. Liu, J. J. Ma, et al. The ML.ENERGY benchmark: Toward automated inference energy measurement and optimization. *arXiv:2505.06371*, 2025.
 - [54] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha. Learned step size quantization. *ICLR*, 2020; arXiv preprint 2019.
 - [55] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer. HAWQ: Hessian aware quantization of neural networks with mixed-precision. *ICCV*, 2019.
 - [56] Z. Dong, Z. Yao, Y. Cai, D. Arfeen, A. Gholami, M. W. Mahoney, and K. Keutzer. HAWQ-V2: Hessian aware trace-weighted quantization of neural networks. *NeurIPS*, 2020.
 - [57] Y. Li, R. Gong, X. Tan, Y. Yang, P. Hu, Q. Zhang, F. Yu, W. Wang, and S. Gu. BRECCQ: Pushing the limit of post-training quantization by block reconstruction. *ICLR*, 2021.
 - [58] Z. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He. ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers. *NeurIPS*, 2022.
 - [59] Z. Yao, X. Wu, C. Li, S. Youn, and Y. He. ZeroQuant-V2: Exploring post-training quantization in LLMs from comprehensive study to low rank compensation. *arXiv:2303.08302*, 2023.
 - [60] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, and P. Luo. OmniQuant: Omnidirectionally calibrated quantization for large language models. *arXiv:2308.13137*, 2024.
 - [61] Z. Liu, B. Oguz, C. Zhao, E. Chang, P. Stock, Y. Mehdad, Y. Shi, R. Krishnamoorthi, and V. Chandra. LLM-QAT: Data-free quantization aware training for large language models. *Findings of ACL*, 2024; arXiv preprint 2023.
 - [62] D. Du, Y. Zhang, S. Cao, J. Guo, T. Cao, X. Chu, and N. Xu. BitDistiller: Unleashing the potential of sub-4-bit LLMs via self-distillation. *arXiv:2402.10631*, 2024.
 - [63] M. Chen, W. Shao, P. Xu, J. Wang, P. Gao, K. Zhang, and P. Luo. EfficientQAT: Efficient quantization-aware training for large language models. *arXiv:2407.11062*, 2024.
 - [64] X. Ma, G. Fang, and X. Wang. LLM-Pruner: On the structural pruning of large language models. *NeurIPS*, 2023.
 - [65] Y. An, X. Zhao, T. Yu, M. Tang, and J. Wang. Fluctuation-based adaptive structured pruning for large language models. *AAAI*, 2024; arXiv preprint 2023.

-
- [66] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Re. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 2022.
 - [67] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *ICLR*, 2024; arXiv preprint 2023.